

Carlos Henrique Mantovani Caliman, Henrique Fuga Duran, João Pedro
Zangelmi Rezende de Menezes

Plataforma para Análise da Produtividade de Equipes com Integração ao GitHub Copilot

São Paulo, SP

2025

Carlos Henrique Mantovani Caliman, Henrique Fuga Duran, João Pedro
Zangelmi Rezende de Menezes

Plataforma para Análise da Produtividade de Equipes com Integração ao GitHub Copilot

Trabalho de conclusão de curso apresentado
ao Departamento de Engenharia de Computa-
ção e Sistemas Digitais da Escola Politécnica
da Universidade de São Paulo para obtenção
do Título de Engenheiro.

Universidade de São Paulo – USP

Escola Politécnica

Departamento de Engenharia de Computação e Sistemas Digitais (PCS)

Orientador: Prof. Dr. João Marcos Barguil

Coorientador: Prof. Dr. Fabio Levy Siqueira

São Paulo, SP

2025

Resumo

O avanço acelerado da inteligência artificial nos últimos anos, especialmente após o lançamento do *ChatGPT*, tem impulsionado a adoção de ferramentas baseadas em IA generativa em diversas áreas, incluindo o desenvolvimento de software. Nesse contexto, destaca-se o *GitHub Copilot*, uma ferramenta que promete aumentar a produtividade de programadores ao sugerir trechos de código em tempo real. No entanto, ainda são escassos os estudos quantitativos que avaliam, de forma sistemática, os reais impactos dessas tecnologias nas rotinas de equipes de desenvolvimento de software. Diante desse cenário, este trabalho propõe o desenvolvimento de um sistema de monitoramento e análise que integra dados de uso do *GitHub Copilot* com informações de repositórios *Git*, permitindo a visualização desses dados por meio de um *dashboard* interativo acessível via *Web*. Com a aplicação do sistema, foi possível identificar padrões que indicam uma relação direta entre o uso das ferramentas de IA generativa e variações na produtividade, oferecendo aos gestores informações valiosas para embasar decisões sobre adoção tecnológica e aprimoramento dos processos de desenvolvimento.

Palavras-chave: *GitHub*, *Git*, *Copilot*, Produtividade.

Abstract

The rapid advancement of artificial intelligence in recent years, especially following the launch of *ChatGPT*, has driven the adoption of generative AI tools across various fields, including software development. In this context, *GitHub Copilot* stands out as a tool that aims to enhance programmer productivity by suggesting code snippets in real time. However, there is still a lack of quantitative studies that systematically assess the real impact of these technologies on software engineering teams' workflows. In response to this gap, this work proposes the development of a monitoring and analysis system that integrates usage data from *GitHub Copilot* with information from Git repositories, enabling the visualization of this data through an interactive web-based dashboard. Through the application of the system, it was possible to identify patterns indicating a direct relationship between the use of generative AI tools and variations in productivity. These insights provide managers with valuable information to support decisions on technology adoption and process improvement in software development.

Keywords: *GitHub*, *Git*, *Copilot*, Productivity.

Sumário

1	INTRODUÇÃO	9
1.1	Objetivos	10
1.2	Justificativa	10
1.3	Organização do Trabalho	10
1.4	Disponibilização da Solução	11
2	ASPECTOS CONCEITUAIS	13
2.1	Produtividade em desenvolvimento de software	13
2.1.1	Métricas de tamanho de código	14
2.2	Sobre correlação de métricas	15
2.2.1	Correlação de <i>Pearson</i>	16
2.2.2	Modelos de Linguagem para Geração de Código e o <i>GitHub Copilot</i>	16
3	MÉTODO DO TRABALHO	19
3.1	Etapas do desenvolvimento	19
3.1.1	Especificação de requisitos	19
3.1.2	Projeto do sistema	20
3.1.3	Implementação	20
3.1.4	Verificação e ajustes	20
4	ESPECIFICAÇÃO DE REQUISITOS	21
4.1	Descrição dos dados extraídos	21
4.2	Métricas de produtividade e correlação com o uso do <i>GitHub Copilot</i>	22
4.2.1	Comparação entre períodos com e sem uso do <i>Copilot</i>	22
4.2.2	Análise da intensidade de contribuição do <i>Copilot</i> na produtividade	22
4.2.3	Correlação entre produtividade e uso da ferramenta generativa	23
4.3	Requisitos Funcionais	23
4.4	Requisitos Não-Funcionais	24
5	DESENVOLVIMENTO DO TRABALHO	27
5.1	Visão geral do Sistema	27
5.2	Descrição da Implementação	29
5.2.1	Backend	29
5.2.1.1	Tecnologias utilizadas no backend	29
5.2.1.2	Domínio	30
5.2.1.3	Consumidores	30

5.2.1.4	Comunicação Externa	31
5.2.1.5	Testes de Unidade	31
5.2.2	Frontend	32
5.2.2.1	Tecnologias utilizadas no frontend	32
5.2.2.2	Rotas	32
5.2.2.3	Componentes	32
5.2.2.4	Serviços	32
5.2.2.5	Tipos	33
5.2.2.6	Constantes	33
5.2.2.7	Hooks	34
5.2.2.8	Páginas	34
5.3	Coleta de dados e processamento de métricas	34
5.3.1	Coleta de dados	34
5.3.1.1	Métricas de <i>Commits</i>	34
5.3.1.2	Métricas do <i>GitHub Copilot</i>	43
5.3.2	Tratamento de dados e cálculo de métricas	47
5.3.2.1	<i>Productivity Analytics</i> - Frontend	47
5.3.2.2	<i>Productivity Analytics</i> - Backend	49
5.3.2.3	Language analytics	53
5.3.2.4	Copilot Metrics	55
5.3.3	Registro e Controle de Dados por Usuário	60
5.3.3.1	Login e Registro	61
5.3.3.2	Proteção de Rotas	64
5.3.3.3	Separação de Dados por Usuário	64
5.3.4	Relatórios periódicos	64
5.3.4.1	Configuração de envio de relatórios	65
5.3.4.2	<i>Worker</i> de envio de relatórios	65
5.3.5	Hospedagem e execução	69
5.4	Discussão	70
5.4.1	<i>Productivity Analytics</i>	71
5.4.2	<i>Language Analytics</i>	71
5.4.3	<i>Copilot Usage Analytics</i>	72
5.4.3.1	<i>Usage Chart</i>	72
5.4.3.2	<i>Suggestion Acceptance Chart</i>	72
5.4.3.3	Análise integrada dos gráficos de <i>Copilot Analytics</i>	73
5.4.4	Análises cruzadas	73
5.4.5	Confidencialidade	73
5.5	Documentação da <i>API</i>	74
6	VALIDAÇÃO	77

6.1	Questionário de Validação do Sistema	77
6.2	Resultados	78
6.2.1	Utilização Com Dados Reais	79
6.2.2	Avaliação Geral dos Usuários	80
6.2.2.1	Perfil dos Respondentes	80
6.2.2.2	Avaliação de Usabilidade e Funcionamento	81
6.2.2.3	Avaliação das Visualizações	81
6.2.2.4	Avaliação Qualitativa	83
6.2.2.5	Avaliação Geral	83
6.3	Resumo	84
7	CONSIDERAÇÕES FINAIS	85
7.1	Visão Geral	85
7.2	Trabalhos Futuros	85
	REFERÊNCIAS	89

1 Introdução

O desenvolvimento de software é uma atividade de alto custo, e qualquer ganho de produtividade pode gerar impactos relevantes em prazos, qualidade e orçamento. [Harness \(2025\)](#) indica que empresas perdem, anualmente, na média, 32 mil dólares em ineficiências de produtividade por desenvolvedor. Diante desse cenário, a busca por métodos e ferramentas que otimizem os processos de Engenharia de Software constitui um desafio para a gestão.

Uma alternativa moderna para endereçar tais ineficiências é a adoção do uso de ferramentas de inteligência artificial (IA) generativa. Desde o lançamento do *ChatGPT* pela *OpenAI* em 2022, o uso de modelos de linguagem de larga escala (comumente abreviadas por *LLMs* - *Large Language Models*) cresceu rapidamente, com milhões de usuários explorando suas aplicações em diversas áreas, incluindo o desenvolvimento de software ([HOFFMAN, 2025](#)). Ferramentas como o *GitHub Copilot*¹, baseadas nesses modelos, têm se destacado ao oferecer sugestões de código, automatizar tarefas repetitivas e auxiliar na documentação, integrando-se diretamente aos ambientes de desenvolvimento (*IDEs*).

O avanço das inteligências artificiais generativas nos últimos anos tem transformado a área, e a adoção dessas ferramentas já demonstra potencial significativo. A *PwC*² reporta um aumento de 20 a 40% na produtividade dos funcionários após a adoção da tecnologia [Weaver \(2024\)](#). Contudo, o investimento em IA generativa pode representar um custo significativo para as organizações. Um estudo de caso envolvendo uma empresa de B2B SaaS (*software as a service, business to business model*) aponta um investimento anual de US\$ 2,1 milhões na adoção de tais soluções pelos seus 3.000 colaboradores [Worklytics \(2025\)](#).

Nesse contexto, ferramentas como o *GitHub Copilot* despertam questões sobre sua real contribuição para a produtividade das equipes de desenvolvimento de Software. Avaliar, de forma quantitativa e qualitativa, esse impacto ainda é um desafio, especialmente para gestores que necessitam embasar decisões sobre a adoção. Além disso, os dados fornecidos pelas próprias plataformas são, em geral, limitados³ e nem sempre atendem às necessidades gerenciais.

Diante desse cenário, este trabalho propõe o desenvolvimento de um sistema de apoio à decisão que possibilite analisar a correlação entre o uso de ferramentas de inteligência artificial generativa e a produtividade no desenvolvimento de software, oferecendo subsídios para decisões estratégicas mais informadas.

¹ Documentação oficial: [GitHub \(2025\)](#).

² Uma dentre as *big four* firmas de auditoria.

³ No caso do *GitHub Copilot*, as informações são bem restritas, como é explicitado no Capítulo 4.

1.1 Objetivos

Este trabalho tem como objetivo projetar e desenvolver um *dashboard* interativo que apresente métricas relevantes sobre o impacto do uso de ferramentas de IA, sobretudo o *GitHub Copilot*, no contexto de desenvolvimento de software.

Para isso, será desenvolvido um sistema de software capaz de:

- Coletar dados brutos de uso do *GitHub Copilot* via sua *API* oficial;
- Coletar dados brutos de repositórios *Git*;
- Armazenar e tratar os dados;
- Apresentar as informações em um *dashboard* dinâmico, acessível via *web*, permitindo visualização das métricas coletadas e como estas influenciam na produtividade de desenvolvimento de *software*.

1.2 Justificativa

O desenvolvimento deste trabalho é relevante para o avanço da compreensão sobre o papel das inteligências artificiais no contexto do desenvolvimento de software. A análise estruturada de métricas de produtividade permitirá que gestores tomem decisões embasadas sobre a adoção dessas tecnologias, avaliando o custo-benefício de sua aplicação em equipes de desenvolvimento, inclusive no que se refere à aquisição de versões *premium* dessas ferramentas.

Justifica-se a realização deste estudo diante da ausência de soluções que apresentem, os efeitos do uso de inteligência artificial generativa sobre a produtividade de desenvolvedores. Embora a adoção dessas tecnologias esteja em expansão, ainda há carência de dados organizados e acessíveis que permitam às organizações avaliar se os ganhos percebidos refletem melhorias reais nos processos de desenvolvimento. Nesse sentido, o sistema proposto neste trabalho busca estruturar os dados provenientes do uso do *GitHub Copilot* e integrá-los com informações dos repositórios de código para gerar visualizações e métricas consolidadas. Essa abordagem tem como objetivo oferecer aos gestores uma visão prática e orientada por dados, auxiliando na tomada de decisões estratégicas relacionadas à adoção e ao uso eficiente de ferramentas baseadas em inteligência artificial.

1.3 Organização do Trabalho

Este documento está estruturado da seguinte forma:

No Capítulo 2, são apresentados os conceitos teóricos utilizados para a medição da produtividade no desenvolvimento de software. Essa fundamentação é essencial para o embasamento da implementação do sistema proposto e a análise dos impactos das inteligências artificiais nesse contexto.

O Capítulo 3 descreve o método do trabalho. Isto é, aqui se descreve como foi o processo de desenvolvimento: além da implementação de código em si (descrita no Capítulo 5), os demais passos referentes a evolução do projeto são contemplados neste capítulo.

O Capítulo 4 detalha os requisitos funcionais e não funcionais do sistema a desenvolvido.

No Capítulo 5, são abordadas as tecnologias utilizadas, a arquitetura do projeto, a sua implementação e a validação do sistema desenvolvido.

Por fim, no Capítulo 7, são apresentadas as conclusões do projeto, suas principais contribuições e as perspectivas de continuidade do trabalho.

1.4 Disponibilização da Solução

Todo o código desenvolvido neste trabalho foi disponibilizado publicamente em um repositório *GitHub* online, disponível em:

<https://github.com/hfduran/canaicode/>

Ademais, a solução pode ser acessada por meio do seguinte link:

<http://canaicode.xyz/>

2 Aspectos Conceituais

Este trabalho adota uma abordagem quantitativa para avaliação da produtividade no contexto do desenvolvimento de software. A proposta se baseia no desenvolvimento de um sistema para a análise de dados objetivos, extraídos de fontes diretamente relacionadas ao processo de desenvolvimento, com o intuito de identificar padrões e variações no desempenho produtivo.

O foco está na utilização de indicadores mensuráveis, que permitam avaliar a produção de forma mais direta, reduzindo a dependência de julgamentos subjetivos ou avaliações qualitativas. Para apoiar essa abordagem, será realizada uma breve fundamentação conceitual com base na literatura especializada, buscando indicar que a produtividade pode ser representada por meio de métricas objetivas em determinados contextos.

Essa base servirá como referência para a escolha dos indicadores adotados, de forma a manter a coerência com práticas comuns no campo da Engenharia de Software.

2.1 Produtividade em desenvolvimento de software

Antes dos anos 2000, [Scacchi \(1994\)](#) já levantava algumas dificuldades na questão de medir a produtividade de desenvolvimento de software. É apontado que esta análise, a princípio, deveria levar em conta na aspectos qualitativos. De tal forma, o pesquisador haveria de englobar métricas como *impacto nos objetivos organizacionais*, isto é: pode-se pensar em índices de satisfação do cliente, retorno sobre o investimento (avaliando em termos financeiros se o ganho de receita é maior que o custo de desenvolvimento e custo de implementação); *colaboração dentro da equipe*, e.g. *code reviews* ou métricas de interação social (mensagens no *chat* do *Slack*¹, *Teams*².); e *índices de satisfação da equipe*, como *eNPS*³ ou *ESI*⁴.

Por outro lado, [Streit e Feye \(2024\)](#) buscam metrificar produtividade no contexto de software de forma mais objetiva. O estudo aponta que, embora as métricas meramente quantitativas não capturem a complexidade dos elementos envolvidos no desenvolvimento de código, elas mantêm uma relevância significativa. Nesse sentido, métricas relacionadas ao tamanho do código (*code size-related metrics*) são consideradas uma ferramenta importante na análise da produtividade de desenvolvimento.

¹ Disponível em: <https://slack.com/intl/pt-br/>.

² Disponível em: <https://www.microsoft.com/pt-br/microsoft-teams/group-chat-software/>

³ O *Employee Net Promoting Score* é uma versão adaptada do NPS - Net Promoting Score, métrica de satisfação do cliente - para o contexto de satisfação do empregado.

⁴ o *Employee Satisfaction Index* é a versão análoga para o empregado do *CSAT* - *Customer Satisfaction Score*.

Dessa forma, neste trabalho, a avaliação de produtividade será encarada com a perspectiva de volume de código produzido, e não se este código desenvolvido atende os requisitos de segurança, escalabilidade, manutenibilidade, confiabilidade, eficiência, portabilidade ou funcionalidade; ou ainda se artifícios externos ao código foram satisfatórios, como aqueles já comentados acima, por [Scacchi \(1994\)](#).

Existem problemas em utilizar métricas meramente referentes ao volume de código. Tais métricas falham em dimensionar a qualidade e a complexidade das *features* construídas, visto que códigos mais eficientes podem atingir o mesmo resultado com menos linhas. Similarmente, um número elevado de *commits* pode indicar entregas fragmentadas, e não, necessariamente, maior volume de desenvolvimento ([GNANASAMBANDAM; HARRYS-SON, 2023](#)). Todavia, apesar de tais critérios não traduzirem perfeitamente o conceito de produtividade, eles ainda se apresentam como métricas valiosas, em especial na análise de um grande volume de dados.

Diante disso, é, primeiramente, necessário entender o que e quais⁵ são as *code size-related metrics*, para que, então, seja possível elencar quais destas seriam utilizadas nas avaliações pretendidas do projeto.

2.1.1 Métricas de tamanho de código

Métricas de tamanho de código (*Code Size-Related Metrics*) são indicadores cuja obtenção é puramente quantitativa, utilizados para mensurar o volume ou a extensão do código-fonte produzido. A aplicação dessas métricas, como a Contagem de Linhas de Código (*LOC - Lines of Code*), foca no volume de código e não se este código desenvolvido atende aos requisitos de segurança, escalabilidade, manutenibilidade, confiabilidade, eficiência, portabilidade ou funcionalidade, ou ainda se artifícios externos ao código foram satisfatórios. Embora o foco principal deste trabalho seja o volume, [Scacchi \(1994\)](#) aponta que algumas destas métricas, mesmo obtidas quantitativamente, conseguem captar elementos relacionados à qualidade de software, fornecendo uma perspectiva de estabilidade do código⁶.

- *Lead Time for Changes (LTFC)*: o tempo tomado entre o código passar de um *commit* para o ambiente de produção;
- *Change Failure Rate (CFR)*: a porcentagem de *deployments* que resultam em falhas em ambiente de produção;
- *Mean Time To Recovery (MTTR)*: tempo médio despendido desde uma falha até a restauração do serviço.

⁵ Não exaustivo.

⁶ Contudo, vale relembrar que esta perspectiva não é o foco do trabalho.

Ademais, [Streit e Feye \(2024\)](#) introduz algumas outras. No estudo, é realizada a análise das seguintes métricas sobre mais de 1 milhão de linhas de código:

- *Changed LoC (Lines of Code)*: o número de linhas adicionadas ou modificadas na *diff*⁷ entre as versões de código no dado período (linhas removidas, mudanças de espaços em branco e linhas vazias não são consideradas, comentários são considerados).
- *RFCC - Redundancy-Free Code Churn*: mede a quantidade de código adicionado ou modificado entre duas versões no dado período de tempo, ajustada para a quantidade de código que foi duplicada ou trocada de lugar.
- *Size LoC Absolute Growth (SLoC AG)*: mede o crescimento absoluto do código, se assemelha a *LoC* mas não ignora linhas removidas.
- *RFSLoC Absolute Growth*: mede o mesmo que *SLoC AG*, mas ajustando para linhas duplicadas.
- *Commits*: o número de *commits* durante o período de tempo analisado.

As métricas utilizadas, mesmo que não levem em conta méritos qualitativos, tem correlação sobre o esforço medido durante o desenvolvimento de software, afirma [Streit e Feye \(2024\)](#). Isto é, os autores concluem que o esforço (medido em horas despendidas, incluindo tarefas como cerimônias ágeis, desenho de arquitetura, etc.) é refletido no resultado da **quantidade** de código produzido, o que será tomado adiante como produtividade.

2.2 Sobre correlação de métricas

Correlação é uma medida estatística que expressa o grau de associação entre duas variáveis. [Zaros e Medeiros \(2011\)](#) afirmam que variáveis quantitativas são consideradas correlacionadas quando sua variação ocorre de forma conjunta, indicando uma associação entre elas. Ademais, é declarado que, em uma correlação positiva, ambas aumentam ou diminuem simultaneamente, enquanto na negativa, variam em direções opostas.

Considerando que o objetivo deste trabalho é investigar a possível relação entre o uso de ferramentas de inteligência artificial generativa e a produtividade no desenvolvimento de software, torna-se pertinente compreender o conceito de correlação. Isso permitirá verificar, de forma objetiva, se existe e qual é a natureza dessa associação entre as variáveis estudadas.

[Bussab e Morettin \(2010, p. 76\)](#) declara:

⁷ A *diff*, abreviação de *difference*, é a comparação entre duas versões de um arquivo ou código, mostrando as diferenças entre elas, exibindo linhas adicionadas, modificadas ou removidas

Essas [coeficientes de correlação] são medidas que descrevem, por meio de um único número, a associação (ou dependência) entre duas variáveis. Para maior facilidade de compreensão, esses coeficientes usualmente variam entre 0 e 1, ou entre -1 e +1, e a proximidade de zero indica falta de associação.

Desta forma, há diversas medidas que quantificam a associação entre duas variáveis. Será apresentado neste trabalho, apenas uma: correlação de *Pearson*.

2.2.1 Correlação de *Pearson*

Como aponta [Zaros e Medeiros \(2011\)](#), o coeficiente de Correlação de *Pearson* para uma amostra de dados pode ser calculado da maneira abaixo, retornando o grau de relação linear entre as variáveis:

$$r = \frac{\sum xy - \frac{\sum x \sum y}{n}}{\sqrt{\left[\sum x^2 - \frac{(\sum x)^2}{n}\right] \cdot \left[\sum y^2 - \frac{(\sum y)^2}{n}\right]}} \quad (2.1)$$

Onde:

r = Coeficiente de Correlação de *Pearson*,

x = Valores da primeira variável,

y = Valores da segunda variável,

n = Número de amostras.

[Bussab e Morettin \(2010, p. 262\)](#) define: "*População* é o conjunto de todos os elementos ou resultados sob investigação [...]. *Amostra* é qualquer subconjunto da população". Na fórmula apresentada, o cálculo do coeficiente de correlação de *Pearson* é direcionado para uma amostra de dados, o que se evidencia pela presença da notação n representando o tamanho da amostra, em contraposição ao tamanho da população.

Na prática, a obtenção desse coeficiente de correlação pode ser realizada através da biblioteca *Pandas* em *Python*, utilizando o método `.corr()`. Conforme detalhado na documentação oficial do *Pandas*⁸, esse método implementa o cálculo da correlação de *Pearson* com base nos dados fornecidos, tratando-os como uma amostra e aplicando as fórmulas estatísticas adequadas para esse contexto, e não para a totalidade de uma população.

2.2.2 Modelos de Linguagem para Geração de Código e o *GitHub Copilot*

Modelos de linguagem de grande porte (*Large Language Models*, ou LLMs) têm sido empregados de forma crescente como forma de apoio ao desenvolvimento de software. Ainda,

⁸ Disponível em: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.corr.html>.

[Dong et al. \(2025\)](#) afirma que agentes de IA de geração de código estão fundamentalmente alterando o ecossistema de Engenharia de Software.

A aplicação de LLMs à geração de código se beneficia de características próprias das linguagens de programação. Linguagens de programação apresentam estrutura altamente regular e sintaxe estrita, favorecendo a aprendizagem destes modelos, uma vez que funcionam ao avaliar sequências de palavras e identificando padrões. Além disso, há ampla disponibilidade de código aberto em plataformas públicas (códigos *open-source*), permitindo que os modelos sejam expostos a um grande número de exemplos reais de implementação (grande disponibilidade de dados), garantindo ao modelo mais oportunidades de aprendizado ([Cisco Outshift, 2024](#)).

O *GitHub Copilot* é uma dessas ferramentas que operacionaliza LLMs no ambiente de desenvolvimento. Por exemplo, no caso do *Copilot*, há integração com a IDE *Visual Studio Code*. O sistema utiliza as IAs para gerar sugestões a partir do contexto local do usuário atuando como um assistente de escrita, oferecendo alternativas que podem ser aceitas, modificadas ou recusadas pelo desenvolvedor, sem substituir as etapas de verificação e validação do código.

Ferramentas baseadas em LLMs têm sido associadas ao aumento da velocidade de desenvolvimento de software. [Tabarsi et al. \(2025\)](#) afirma qualitativamente que assistentes de código tornam desenvolvedores mais rápidos, ao passo que tais ferramentas podem reduzir o tempo necessário para implementar funcionalidades e acelerar tarefas repetitivas ou estruturais, contribuindo para ciclos de desenvolvimento mais curtos. Nesse sentido, a integração entre modelos de linguagem e ambientes de programação representa uma mudança de paradigma nas etapas do processo de desenvolvimento de software.

3 Método do trabalho

O desenvolvimento deste trabalho foi conduzido por meio de um processo estruturado em fases, com entregas parciais de funcionalidades que foram progressivamente ampliadas até a obtenção do *Minimum Viable Product* (*MVP*, ou Produto Mínimo Viável). Após essa etapa inicial e a validação, o sistema é aprimorado com a inclusão de novas funcionalidades que agregam valor ao usuário. Esta seção tem como objetivo apresentar as etapas e o cronograma de desenvolvimento do projeto. O escopo das funcionalidades do sistema, o detalhamento das tecnologias utilizadas, a explicitação do software desenvolvido e as análises e interpretações dos dados obtidos, serão temas abordados nos capítulos seguintes.

3.1 Etapas do desenvolvimento

As etapas percorridas ao longo da implementação seguiram uma lógica sequencial voltada à construção progressiva e validada do sistema, desde a definição dos requisitos até a entrega do *dashboard* funcional. Inicialmente, foram analisadas as métricas de produtividade disponíveis e definidas as funcionalidades básicas do sistema. Durante o desenvolvimento, novas visualizações dos dados foram incorporadas, até alcançar o *MVP*. Na etapa seguinte, foram adicionadas funcionalidades complementares e investigadas métricas adicionais de produtividade que pudessem agregar valor ao produto.

A seguir, cada uma das fases será apresentada com a devida contextualização dentro do processo de desenvolvimento adotado para este trabalho.

3.1.1 Especificação de requisitos

A especificação dos requisitos do sistema ocorreu de forma contínua e iterativa ao longo do desenvolvimento do projeto, acompanhando a evolução das descobertas técnicas, da disponibilidade dos dados e do próprio escopo da solução. A definição dos requisitos foi sendo ajustada gradualmente à medida que novas necessidades surgiam e o entendimento do problema se aprofundava. Esse processo envolveu discussões frequentes com os orientadores do trabalho, que atuaram como *stakeholders* ativos. Em especial, um dos orientadores tem relação profissional em uma empresa de Engenharia de Software, o que lhe permitiu contribuir com ideias sobre funcionalidades que poderiam agregar valor em contextos reais de uso, assim, assumindo uma perspectiva de um possível usuário final.

Os requisitos são divididos entre funcionais — que descrevem o comportamento do sistema — e não funcionais — que, neste projeto, definem primordialmente características

de usabilidade — e estão descritos no Capítulo 4.

3.1.2 Projeto do sistema

Com os requisitos estabelecidos, inicia-se a etapa de projeto do sistema, responsável por definir a arquitetura geral da solução. Isso inclui decisões sobre a estrutura de software, tecnologias que serão utilizadas, modelos de dados, organização dos módulos e a forma de integração entre os componentes. Ademais, é definida a estrutura do banco de dados, os pontos de integração com *APIs* externas, e o fluxo de dados entre os elementos do sistema.

As decisões tomadas acerca das escolhas de projeto postas acima, são descritos no Capítulo 5.

3.1.3 Implementação

A fase de implementação consiste na construção efetiva do sistema, com base nas definições feitas na etapa de projeto. O desenvolvimento foi dividido em módulos, respeitando uma organização incremental, com entregas parciais que permitiram validações contínuas ao longo do processo. Nessa fase, foi implementado o código responsável pela coleta dos dados via *API* do *GitHub Copilot*, extração de dados dos repositórios *Git*, persistência das informações no banco de dados, além do próprio *dashboard web*.

Novamente, os processos referidos acima são descritos em sua íntegra no Capítulo 5.

3.1.4 Verificação e ajustes

Ao longo de todo o processo de desenvolvimento do *dashboard*, a verificação da sua qualidade e adequação aos objetivos foi uma atividade contínua. Isso envolveu a realização conjunta de testes em diferentes níveis: a verificação isolada de componentes com testes de unidade e a avaliação do comportamento geral da aplicação através de testes de sistema.

Ao passo que estas atividades contínuas de verificação técnica se desdobraram, esta interação constante permitiu identificar inconsistências, oportunidades de melhoria e a necessidade de ajustes para garantir que o sistema funciona de acordo com o atendimento dos objetivos propostos.

4 Especificação de Requisitos

Neste capítulo, são descritos o escopo da aplicação, as fontes de dados utilizadas e a forma como as informações tratadas são visualizadas. Em seguida, são detalhados os requisitos funcionais, que descrevem os comportamentos e funcionalidades esperadas do sistema, e os requisitos não funcionais, que estabelecem critérios de qualidade que o sistema deve atender.

4.1 Descrição dos dados extraídos

O sistema é responsável por coletar e armazenar dados provenientes de duas fontes principais: a *API* oficial do *GitHub Copilot* e repositórios *Git* utilizados no desenvolvimento dos projetos analisados.

A coleta de dados a partir da *API* do *GitHub Copilot* visa obter métricas sobre a interação dos desenvolvedores com a ferramenta. São disponibilizados os dados:

- Total de usuários ativos;
- Total de usos, usuários, eventos de cópia e eventos de inserção do chat da ferramenta, segmentados por IDE, modelo e dia;
- Número de usuários, sugestões geradas, sugestões aceitas, linhas sugeridas e linhas aceitas em complementação de código, segmentados por IDE, modelo, linguagem e dia.

A coleta dos dados do *GitHub Copilot* é feita de duas maneiras: periodicamente, com base em uma rotina automatizada ou manualmente pelo usuário, que faz o *upload* do arquivo *json* na plataforma *WEB*.

Paralelamente, é feita a extração de dados brutos de repositórios *Git* vinculados aos projetos estudados. Essa etapa envolverá o uso de ferramentas de linha de comando, como *git log*, para análise programática dos repositórios. As informações extraídas incluirão:

- total de linhas inseridas;
- total de linhas removidas;
- linguagem de programação utilizada;
- data do *commit*;

- autor do *commit*.

Essas informações são utilizadas para compor o histórico de desenvolvimento e permitir a análise cruzada com os dados do *Copilot*. Novamente, a coleta pode ocorrer de duas formas: automaticamente, via configuração do *GitHub Actions* ou manualmente pelo usuário, que utiliza o *script* disponibilizado para exportar os dados em um arquivo *XLSX* e faz o *upload* do arquivo na plataforma.

Após sua coleta, todos os dados são devidamente armazenados e preparados para visualização, bem como novas métricas calculadas a partir destes dados brutos. O objetivo é garantir que essas informações estejam organizadas de forma estruturada, possibilitando sua posterior apresentação no *dashboard* interativo.

4.2 Métricas de produtividade e correlação com o uso do *GitHub Copilot*

Como detalhado no Capítulo 2, o presente trabalho adota uma abordagem quantitativa para avaliar a produtividade no contexto de desenvolvimento de software. A produtividade, nesse caso, é entendida como a quantidade de código produzido em determinado período de tempo. Com base nas conclusões de Streit e Feye (2024) e na gama de dados disponíveis, optou-se por utilizar duas métricas principais: *Changed Lines of Code* (*Changed LoC*) e *Commits*, ambas obtidas diretamente da análise dos repositórios.

Para correlacionar a produtividade ao uso do *GitHub Copilot*, diferentes abordagens complementares são aplicadas.

4.2.1 Comparação entre períodos com e sem uso do *Copilot*

Decorre de uma análise temporal da produtividade, comparando os indicadores de vazão de *Changed Lines of Code* e *Commits* em períodos distintos, classificando-os conforme a presença ou ausência do uso da ferramenta. Essa comparação permite identificar tendências ou alterações significativas no volume de código produzido quando a ferramenta é adotada.

4.2.2 Análise da intensidade de contribuição do *Copilot* na produtividade

Além da comparação por períodos, também é avaliado o quanto o *GitHub Copilot* contribuiu diretamente para o resultado de *Changed LoC* em um período. Isso é possível através da identificação da quantidade de linhas aceitas nas sugestões da ferramenta nesse período, permitindo estimar a proporção de linhas adicionadas que foram escritas pelo

Copilot. Essa análise é apresentada de forma tanto absoluta quanto percentual, oferecendo uma visão mais detalhada sobre o impacto da ferramenta no processo produtivo.

4.2.3 Correlação entre produtividade e uso da ferramenta generativa

Por último, outro elemento exposto para o usuário é a quantificação da relação entre o uso da ferramenta generativa e a produtividade, realizada por meio do coeficiente de Correlação de *Pearson*. O objetivo é demonstrar, da forma mais direta possível, em um único número, se existe de fato uma associação linear do uso do *GitHub Copilot* com a produtividade no desenvolvimento. Ademais, como é explicado adiante, o valor visualizado irá refletir o resultado calculado de acordo com a seleção de filtros escolhidos, de forma que, para cada cenário construído, pode-se haver ou não correlação entre as duas métricas.

4.3 Requisitos Funcionais

RF.1 Coleta de dados do *GitHub Copilot* via *API*:

O sistema deve ser capaz de autenticar-se na *API* oficial do *GitHub Copilot* e coletar dados brutos relacionados ao uso da ferramenta. Serão obtidos o número de sugestões geradas e aceitas, além da quantidade de linhas de código que foram sugeridas e acatadas, como explicitado na Seção 4.1.

RF.2 Coleta de dados de um repositório *Git*:

O sistema deve extrair métricas de um repositório *Git*: número de *commits*, número de linhas, data, autor e linguagem de programação do *commit*, como explicitado na Seção 4.1. Essa extração pode ser automatizada via *GitHub Actions* ou realizada manualmente pelo usuário utilizando um *script* disponibilizado.

RF.3 *Upload* de dados brutos¹:

O sistema deve permitir o *upload* de dados brutos para caso o usuário deseje coletá-los manualmente. Deve ser capaz de processar os dados do *GitHub Copilot* enviados em um arquivo *json* no mesmo formato do retorno da *API* oficial e os dados de um repositório *Git* enviados em um arquivo *XLSX* no formato retornado pelo *script* disponibilizado.

¹ Os itens "Coleta de dados de um repositório *Git*" e "Upload de dados brutos" demonstram as possíveis abordagens para a coleta de dados que o usuário dispõe. Estas abordagens são: a extração automática, que requer a configuração de credenciais de acesso ao repositório *Git* (via *GitHub Actions*); e a inserção manual dos dados. A opção de inserção manual é fornecida para usuários que optam por não disponibilizar as credenciais do repositório à plataforma por questões de segurança, preferindo realizar a coleta localmente (*e.g.* utilizando o *script* disponibilizado) e efetuar o *upload* dos arquivos brutos.

RF.4 Armazenamento de dados:

O sistema deve armazenar os dados coletados em um banco de dados estruturado, preservando tanto os dados brutos quanto os dados tratados.

RF.5 Cálculo de métricas:

O sistema deve realizar o tratamento dos dados e analisá-los, calculando as métricas de *Changed LoC (Lines of Code)* e *commits* para o período selecionado. Além disso, o sistema deve explicitar a correlação de *Pearson* entre as variáveis produtividade total e produtividade do *GitHub Copilot*, de acordo com a seleção da métrica.

RF.6 Exibição de métricas em um *dashboard web* interativo:

O sistema deve apresentar os dados analisados e métricas calculadas em um *dashboard* dinâmico, com visualizações gráficas. O *dashboard* é dividido em seções com diferentes propostas, para apresentar tanto as métricas calculadas quanto os dados brutos de uso do *GitHub Copilot*, como descrito no Capítulo 5.

RF.7 Dinamicidade do *dashboard*:

O sistema deve permitir que o usuário filtre as métricas exibidas de acordo com suas demandas:

- a) Seleção por periodicidade (semana, mês, trimestre, ano)
- b) Filtro de data (início e fim do período considerado)
- c) Métrica de produtividade observada
- d) Filtro por linguagem de programação
- e) Filtro de metrificar por número de envolvidos

RF.8 Exportação de relatórios:

O sistema deve permitir a configuração de um envio periódico de um arquivo em formato *CSV* ou *XLSX* com métricas referentes ao último período para o *email* do usuário.

RF.9 *Login* e autenticação por usuário:

O sistema deve ser capaz de permitir o registro de novos usuários e o login de usuários registrados.

RF.10 Segmentação de dados por usuário:

Cada usuário deve ter acesso somente aos próprios dados.

4.4 Requisitos Não-Funcionais

RNF.1 Segurança de acesso via OAuth 2.0:

Todas as APIs do sistema tem segurança garantida a nível de *token OAuth 2.0*.

RNF.2 Atualização periódica dos *dashboards*:

No caso da opção pela coleta automática dos dados, o sistema deve atualizar periodicamente os *dashboards* sem a necessidade de intervenção do usuário.

RNF.3 Compatibilidade entre navegadores:

O *dashboard* deve funcionar corretamente nos quatro navegadores mais populares – *Chrome*, *Firefox*, *Edge*, *Safari*.

RNF.4 Prevenção de duplicidade de registros:

O sistema deve impedir que dados já armazenados sejam regravados, realizando uma verificação de integridade sempre que novos dados forem enviados.

RNF.5 Coleta de dados sem acesso a repositórios:

O sistema deve permitir a extração das informações necessárias sem exigir acesso aos repositórios da organização, garantindo que nenhum dado confidencial seja exposto durante o processo.

5 Desenvolvimento do Trabalho

Este capítulo apresenta o processo de desenvolvimento do sistema proposto, abordando desde as tecnologias empregadas até a descrição de sua estrutura e funcionamento. São discutidos os principais componentes que compõem a solução, bem como as decisões técnicas adotadas ao longo de sua implementação, de forma a permitir uma compreensão clara sobre o processo de construção do sistema.

5.1 Visão geral do Sistema

O sistema é composto por quatro camadas principais: *frontend*, *backend*, lógica de negócio e banco de dados. O *frontend*, fornece a interface gráfica que permite o *upload* de arquivos (JSON e XLSX) e a visualização das métricas no *dashboard*. Esses arquivos são enviados ao *backend*, que expõe *endpoints REST* responsáveis por autenticação, recepção de arquivos e requisição de análises. Dentro do *backend*, a camada de lógica de negócio contém dois módulos principais: o *File Parser*, que interpreta e armazena os dados brutos no banco, e o *Metrics Calculator*, que processa os dados e calcula métricas sobre estes. Por fim, o banco de dados (PostgreSQL) armazena as tabelas apenas com os dados brutos. Na Figura 1, abaixo, o diagrama demonstra o racional descrito acima.

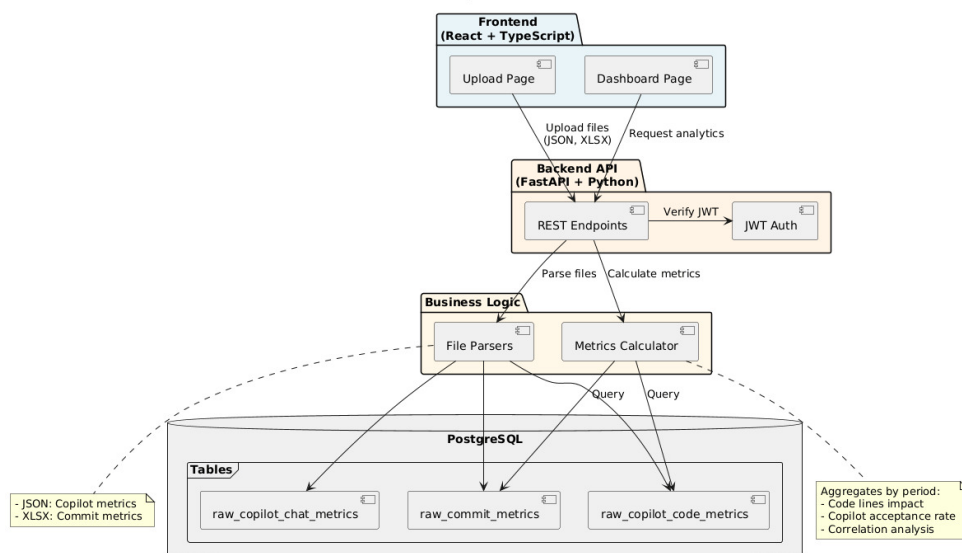


Figura 1 – Diagrama de alto nível da arquitetura do sistema.

Partindo de uma perspectiva de utilização do sistema, o ator principal é o usuário que, após realizar seu registro e autenticação (*login*), acessa as funcionalidades de carregamento e visualização analítica de seus dados.

Os principais cenários de uso são descritos a seguir:

- Registrar-se no sistema: o usuário fornece suas informações cadastrais (nome, e-mail, senha, CPF/CNPJ, entre outros) para criar uma conta.
- Realizar *login*: o usuário insere suas credenciais para acessar o sistema, sendo autenticado e autorizado a interagir com suas próprias informações.
- Carregar dados: após o *login*, o usuário pode realizar o *upload* de arquivos contendo métricas de *commits* e métricas de uso do *Copilot*. Os dados são processados e armazenados nas tabelas correspondentes no banco de dados.
- Analisar dados: o usuário interage com os *dashboards*, que oferecem diferentes visões e filtros para a exploração dos dados.

Em todas as jornadas relacionadas ao *dashboard* e *upload* de métricas, ocorre autenticação a nível de usuário, garantindo que o usuário visualize apenas os seus próprios dados.

A seguir, é apresentado na Figura 2 o diagrama de sequência, que descreve o fluxo de interação entre o usuário e os principais componentes do sistema durante o processo de visualização das métricas no *dashboard*.

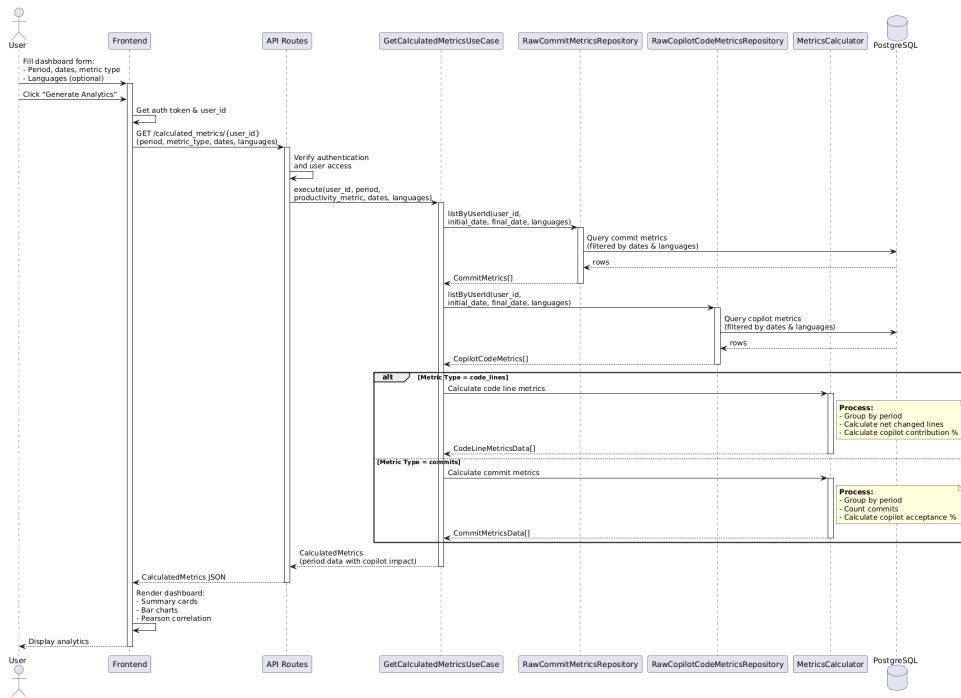


Figura 2 – Diagrama de sequência: visualização de métricas.

5.2 Descrição da Implementação

Esta seção tem por objetivo descrever os pormenores da implementação do sistema, detalhando as decisões técnicas adotadas para o desenvolvimento do projeto.

5.2.1 Backend

5.2.1.1 Tecnologias utilizadas no backend

O *Backend* do sistema foi desenvolvido com uma arquitetura em camadas, de forma a promover a separação de responsabilidades, facilitar a manutenção e garantir a escalabilidade da aplicação. Essa divisão contempla três principais camadas: Domínio, Consumidores e Comunicação Externa. A camada de Domínio concentra as entidades e a lógica de negócio da aplicação; a de Consumidores é responsável pelo processamento e adequação das métricas brutas de *commits* e do *GitHub Copilot*; e a de Comunicação Externa gerencia a interação com o banco de dados, a disponibilização da API e a execução de tarefas periódicas. Por fim, há uma camada adicional destinada aos testes de unidade.

Para o desenvolvimento deste projeto, utiliza-se a linguagem de programação *Python*, devido a sua ampla utilização no contexto de análise de dados, ([DEEPLARNINGAI, 2024](#)), além da familiaridade dos integrantes do grupo com a linguagem.

Para a análise e tratamento de dados, utiliza-se a combinação de duas bibliotecas consolidadas no mercado: *Pandas* e *Numpy*. A primeira é uma biblioteca que tem como função primordial lidar com dados tabulares, enquanto a segunda possibilita fazer diversos tipos de cálculos estatísticos e com precisão arbitrária, com diferentes formatos de dados (sobretudo, tabulares provindos do *Pandas*).

A respeito das ferramentas mais específicas à gestão de qualidade de projeto e de software, utilizam-se as seguintes:

- ***Pipenv***: uma ferramenta que possibilita distribuições determinísticas do software, encapsulando dependências.
- ***MyPy***: trata-se de um analisador sintático do *Python*. Com esta ferramenta é possível verificar tipos das variáveis no código.
- ***Ruffus***: ferramenta para análise de boas práticas de código e formatação.
- ***Pytest***: biblioteca utilizada para testes de unidade e análise de cobertura de teste no código.

Além disso, um *gitflow* foi estabelecido para o desenvolvimento do projeto. *Branches* de desenvolvimento são originadas da *branch* principal, a qual ao efetuar *merge*

executa tarefa de análise sintática e testes de unidade, a fim de avaliar a saúde do código desenvolvido.

5.2.1.2 Domínio

A camada de Domínio representa o núcleo da aplicação, reunindo os elementos centrais que descrevem as regras e abstrações do sistema. Nessa camada estão definidas as entidades, como `User` e `RawCommitMetrics`, que correspondem diretamente às tabelas do banco de dados. Além das entidades, a camada inclui objetos de valor, estruturas auxiliares utilizadas pelas entidades, mas que não possuem representação própria no banco, como as classes `Author` e `Repository`. O trecho de código, na Figura 3, mostra a declaração da entidade que representa o usuário.

```
1 class User(Entity):
2     username: str
3     hashed_password: str
4     full_name: str
5     enterprise_name: Optional[str] = None
6     email: str
7     cellphone: str
8     cpf_cnpj: str
9     created_at: Optional[datetime] = None
```

Figura 3 – Definição da entidade `User`

Para reforçar a consistência e segurança de tipos, também são empregados enumeradores que definem conjuntos limitados de valores, como os períodos de análise e as categorias de métricas de produtividade.

Por fim, essa camada abriga a lógica de negócio, onde são implementadas as regras centrais da aplicação. Nela estão inclusas operações como a criação de usuários, registro de *GitHub Apps*, gerenciamento de chaves de *API*, geração sob demanda de métricas e obtenção de métricas brutas. Essa camada é totalmente isolada de detalhes de infraestrutura, concentrando-se unicamente nas regras e comportamentos esperados do sistema.

5.2.1.3 Consumidores

A camada de Consumidores é responsável por processar e adaptar as informações brutas coletadas dos repositórios. Sua função principal é realizar o *parse* e adequação das métricas brutas, tanto em cenários de extração automática, executada periodicamente, quanto em *uploads* manuais realizados pelo usuário.

Essa camada atua como intermediária entre a coleta de dados e a lógica de negócio, garantindo que as informações provenientes de fontes externas — como o histórico de *commits* e as métricas do *GitHub Copilot* — sejam corretamente estruturadas e

padronizadas antes de serem persistidas no banco de dados ou utilizadas na geração de análises.

5.2.1.4 Comunicação Externa

A camada de Comunicação Externa reúne os componentes responsáveis pela interação do sistema com agentes externos, sendo composta por três subcamadas: *API*, *Scheduler* e Infraestrutura de Banco de Dados.

- *API*: Desenvolvida com o *framework FastAPI*, essa subcamada define as rotas de comunicação com o sistema, sendo responsável por receber e responder às requisições do *frontend*. Nela são realizadas tarefas como a criação e gerenciamento das sessões de banco de dados, a validação da autenticação dos usuários, a injeção de dependências e a chamada dos serviços da camada de aplicação.
- *Scheduler*: Utilizando a biblioteca *APScheduler*, essa subcamada define e gerencia a execução de tarefas periódicas, como a coleta automática de métricas. Assim como na *API*, cada tarefa cria sua própria sessão de banco de dados e injeta as dependências necessárias antes de acionar a lógica de negócio correspondente.
- *Infraestrutura de Banco de Dados*: Essa subcamada é responsável pela configuração da conexão com o banco de dados e pela implementação dos repositórios, que encapsulam as operações de persistência. Para cada tabela do banco, há um repositório que define o modelo de dados, o mapeamento entre o modelo e a entidade de domínio, e os métodos de manipulação (criação, listagem, atualização e exclusão). O *Objeto Relational Mapping* (ORM) escolhido foi o *SQLAlchemy*, por sua compatibilidade e integração natural com o *FastAPI*, além de sua flexibilidade na definição de mapeamentos e consultas.

5.2.1.5 Testes de Unidade

Durante o desenvolvimento das funcionalidades do sistema, foram implementados testes de unidade para a garantia do correto funcionamento. Os testes de unidade são essenciais porque permitem verificar, de forma rápida e isolada, se cada parte do sistema funciona corretamente, oferecendo uma rede de segurança que reduz regressões e aumenta a confiabilidade do software. Além disso, eles incentivam a escrita de código mais modular, simples e bem estruturado, pois somente componentes coesos e independentes são facilmente testáveis.

Com uma base sólida de testes, a equipe ganha confiança para refatorar e evoluir o sistema sem medo de quebrar funcionalidades existentes, acelerando o ciclo de desenvolvimento e depuração.

A *stack* utilizada para efetuar os testes é através da biblioteca *pytest*. Com esta biblioteca, é possível escrever testes com desempenho e parametrizáveis, além de ser possível emitir relatório de cobertura de códigos a partir das métricas de linhas de código cobertas, e branches cobertas.

A meta da equipe foi de manter a cobertura de testes de ambas as métricas acima 80%, o que garante um bom equilíbrio entre cobertura e agilidade de desenvolvimento.

5.2.2 Frontend

5.2.2.1 Tecnologias utilizadas no frontend

Para o desenvolvimento da interface, optou-se pelo *React.js*, uma biblioteca de *JavaScript*. A escolha é fundamentada pela sua ampla adoção na indústria e a familiaridade dos integrantes do presente projeto com a mesma.

No que diz respeito a construção dos componentes visuais, utilizou-se a biblioteca *Cloudscape*¹, da Amazon. Esta oferece um conjunto de elementos de *UI* prontos para uso, garantindo consistência ao *design*.

A visualização dos dados em gráficos é feita com *Recharts*². A biblioteca referida permite a criação de gráficos flexíveis e personalizáveis, elementos essenciais para a construção do *dashboard* em questão.

5.2.2.2 Rotas

As rotas são o que associam páginas a rotas da *URL*, permitindo *links* entre uma página e outra. No projeto, foram diferenciadas rotas públicas de rotas protegidas. As rotas públicas podem ser acessadas por um usuário não cadastrado, enquanto que as protegidas exigem autenticação para serem mostradas - estes pormenores são explicitados mais à frente, na Seção 5.3.3.

5.2.2.3 Componentes

A fim de facilitar o desenvolvimento de funcionalidades presentes em mais de uma tela, fez-se uso da componentização de código, isto é, a utilização de componentes modulares e reutilizáveis, tratando-os como blocos de construção independentes.

5.2.2.4 Serviços

Os Serviços no frontend são módulos encapsulados responsáveis por orquestrar a comunicação assíncrona com a camada de backend por meio de requisições à *API*.

¹ Disponível em: <https://cloudscape.design/>.

² Disponível em: <https://recharts.github.io/>

Essa arquitetura garante a separação de responsabilidades na camada de apresentação, existindo um Serviço específico para gerenciar as interações de cada módulo funcional da aplicação (*e.g.*, um serviço dedicado para *Upload* de métricas, outro para autenticação e registro de usuários, etc.).

5.2.2.5 Tipos

Outro elemento relevante na organização do diretório `src` do *frontend* refere-se à definição de *tipos*. Esses tipos são estruturados em duas categorias principais — *model indexes* e *ui indexes* — reunidas no diretório `types`. Essa separação tem como finalidade explicitar as estruturas de dados utilizadas pelo sistema, tanto no nível da interface quanto na comunicação com o *backend*.

- **Tipos de *UI*:**

Correspondem às interfaces utilizadas para compor os elementos básicos da camada de apresentação, como `DropdownButtonProps`, `SelectOption` e `InputFormProps`. Essas definições orientam a implementação dos componentes visuais, assegurando uniformidade na manipulação das propriedades necessárias ao funcionamento da interface gráfica.

- **Tipos de *Model*:**

Englobam as interfaces que descrevem as estruturas de dados recebidas pelo *frontend* a partir da *API*. Essas definições garantem que os objetos manipulados pela aplicação estejam alinhados aos contratos estabelecidos pelo *backend*.

Na figura 4, apresenta-se um exemplo representativo dessa estrutura: a interface `CalculatedMetricsRequest`.

```
1 export interface CalculatedMetricsRequest {  
2   team_name?: string;  
3   period: "W" | "M" | "Q" | "Y"; // Week, Month, Quarter, Year  
4   productivity_metric: "code_lines" | "commits";  
5   initial_date: Date;  
6   final_date: Date;  
7   programming_languages?: string[];  
8 }
```

Figura 4 – Interface `CalculatedMetricsRequest` utilizada pelo frontend

5.2.2.6 Constantes

A camada de *frontend* também inclui um conjunto de constantes que padronizam valores utilizados de forma recorrente em diferentes partes da aplicação.

Um exemplo central é o conjunto de linguagens de programação suportadas pela aplicação. Essas linguagens são definidas no arquivo de constantes e utilizadas tanto para compor elementos de interface — como seletores e filtros — quanto para validar entradas e estruturar requisições enviadas ao *backend*. A definição é acompanhada de um tipo derivado, que assegura que apenas valores pertencentes à lista pré-definida possam ser utilizados ao longo do código, reforçando a segurança tipada da aplicação.

5.2.2.7 Hooks

Hooks são formas de encapsular lógica reativa para reaproveitamento e melhor separação de responsabilidades. Durante o processo de desenvolvimento, vários hooks foram criados. Um hook tipicamente é responsável por coordenar uma variedade de estados, a fim de que a parte do código que o chame não precisa se preocupar com eles, e use apenas os métodos e estados de sua interface.

5.2.2.8 Páginas

Por fim, as páginas são os componentes associados às rotas, que conterão o layout de um acesso de um usuário a uma URL específica. Uma página é tipicamente formada por uma conjunção de componentes, hooks e código próprio.

5.3 Coleta de dados e processamento de métricas

Esta seção aborda o percurso completo dos dados ao longo do sistema, evidenciando como cada etapa foi concebida e desenvolvida para garantir a transformação adequada das informações desde a origem até sua apresentação final ao usuário. O texto apresenta o fluxo de processamento sob uma perspectiva técnica, evidenciando as principais decisões de implementação que orientaram a construção de cada etapa do sistema.

5.3.1 Coleta de dados

A análise proposta neste trabalho requer a integração de dois conjuntos de dados: métricas de uso da ferramenta *GitHub Copilot* e dados de *commits* extraídos do repositório *Git*.

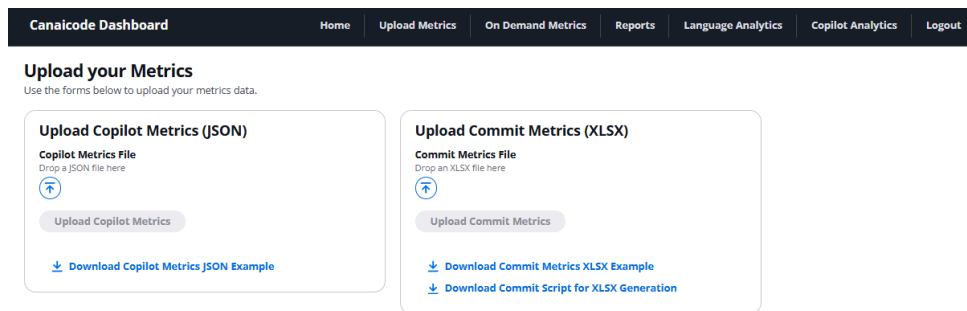
5.3.1.1 Métricas de *Commits*

As métricas de *commits* representam o conjunto de informações extraídas dos repositórios *Git* que descrevem a atividade de desenvolvimento ao longo do tempo. A estrutura básica desses dados pode ser observada na Tabela 1, abaixo, que apresenta um exemplo do formato bruto utilizado como entrada no sistema.

Tabela 1 – Amostra de Dados de *Commits* Extraídos do Repositório *Git*

Hash	Repository	Date	Author	Language	Added Lines	Removed Lines
hash123456	repo	2025-08-01 00:00:00	user _A @github.com	Python	23	21
hash654321	repo	2025-08-01 00:00:00	user _B @github.com	Java	55	32

Foram implementadas duas formas de inserção dessas informações no sistema, sob a perspectiva do usuário: uma forma manual e uma forma automática, conforme ilustrado na Figura 5. Essa flexibilidade visa contemplar diferentes contextos de uso, permitindo tanto o carregamento direto de arquivos exportados pelo usuário quanto a integração contínua com repositórios hospedados no GitHub.

Figura 5 – Tela de *upload* de dados.

(1) Forma manual.

Na modalidade manual, o usuário realiza o carregamento de um arquivo contendo as métricas de *commits* por meio da interface de *upload* de métricas. Essa interface, representada na Figura 5, permite que o arquivo seja selecionado e enviado diretamente ao servidor.

Para os casos em que o usuário opta pela coleta manual dos dados de *commits*, foi desenvolvido um *script* em *Python* que automatiza a extração das métricas a partir de um ou mais repositórios *Git*. Esse script deve ser executado localmente pelo usuário, gerando um arquivo no formato *.xlsx*, que contém, para cada repositório analisado, uma aba com os registros correspondentes. O arquivo resultante é compatível com o formato esperado pelo sistema (Tabela 1) e pode ser submetido diretamente por meio da interface de *upload* citada anteriormente.

O funcionamento do *script* baseia-se na leitura de um arquivo texto contendo as *URLs* dos repositórios *Git* a serem processados, juntamente com um intervalo de datas definido pelo usuário. Para cada repositório listado, o programa realiza o *clone* temporário do projeto, extrai todos os *commits* compreendidos no intervalo informado e contabiliza, para cada arquivo alterado, o número de linhas adicionadas e removidas. As informações extraídas incluem o identificador do *commit*, o autor, a data, a linguagem de programação predominante e o nome do repositório.

A estrutura central do código encontra-se encapsulada na classe `GitRepoConsumer`, responsável por realizar a iteração sobre os *commits* e extrair os dados associados. O trecho de código, na Figura 6, exemplifica o método principal utilizado para essa coleta:

```

1 def get_commits_by_date(self, date: date) -> List[CommitMetrics]:
2     result: List[CommitMetrics] = []
3
4     for commit in self.repo.iter_commits("--all"):
5         commit_date = datetime.fromtimestamp(commit.committed_date).
6             date()
7         if commit_date != date:
8             continue
9
10        if not commit.parents:
11            diff = self.repo.git.diff(self.__NULL_TREE, commit.
12                hexsha, "--numstat")
13        else:
14            diff = self.repo.git.diff(
15                commit.parents[0].hexsha, commit.hexsha, "--numstat"
16            )
17
18        author: str | None = commit.author.email
19
20        for line in diff.splitlines():
21            parts = line.strip().split("\t")
22            if len(parts) != 3:
23                continue
24
25            added, removed, filename = parts
26            if added == "-":
27                added = 0
28            if removed == "-":
29                removed = 0
30
31            language = self.__get_language(filename)
32            result.append(
33                CommitMetrics(
34                    added_lines=int(added),
35                    author=author,
36                    date=datetime.combine(commit_date, time.min),
37                    hash=commit.hexsha,
38                    language=language,
39                    removed_lines=int(removed),
40                    repository=os.path.basename(os.path.normpath(
41                        self.repo_path)),
42                )
43            )
44
45        return result

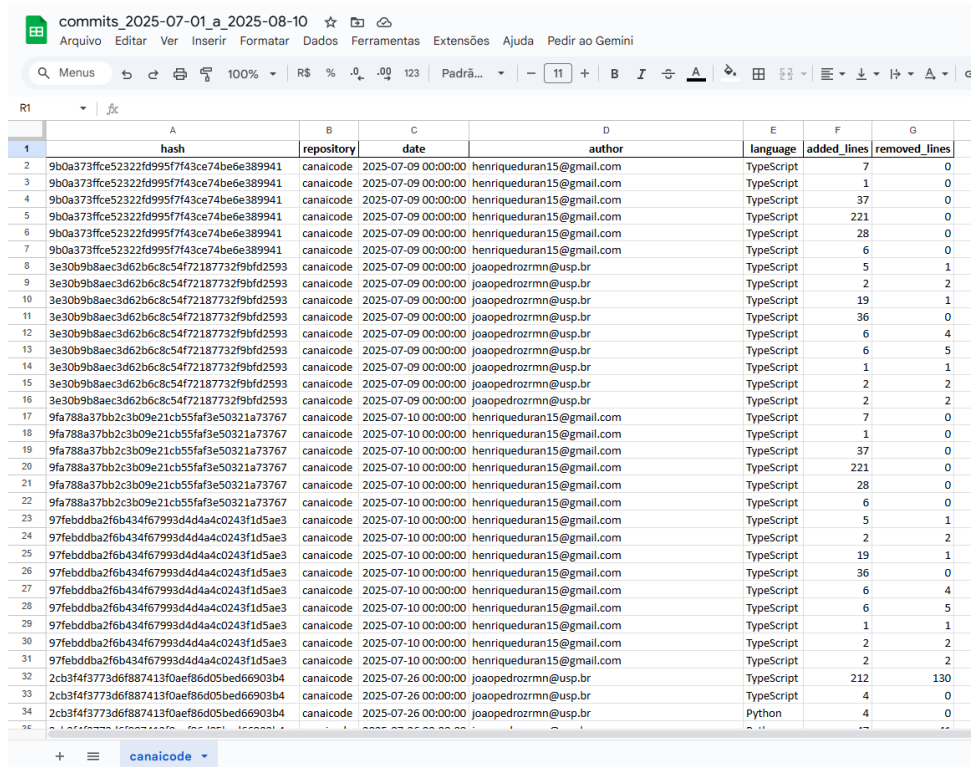
```

Figura 6 – Trecho do método de coleta de *commits* do script manual

Esse método percorre todos os *commits* do repositório e, para cada um, realiza uma comparação entre o estado atual e o anterior, utilizando o comando `git diff` com a opção `-numstat`. Essa operação permite obter, para cada arquivo modificado, o número de linhas adicionadas e removidas. A função também identifica a linguagem de programação

com base na extensão do arquivo, utilizando um mapeamento definido internamente na classe.

O resultado da execução do *script* é um arquivo consolidado, nomeado conforme o intervalo de datas informado, assim como é documentado no arquivo `README`, também provisionado ao usuário na tela de 5. Cada aba do arquivo (*.xlsx*) representa um repositório distinto, contendo as devidas colunas `hash`, `repository`, `date`, `author`, `language`, `added_lines` e `removed_lines`.



	A	B	C	D	E	F	G
	hash	repository	date	author	language	added_lines	removed_lines
1	9b0a373ffce52322fd995f7f43ce74be6e389941	canalcode	2025-07-09 00:00:00	henriqueduran15@gmail.com	TypeScript	7	0
3	9b0a373ffce52322fd995f7f43ce74be6e389941	canalcode	2025-07-09 00:00:00	henriqueduran15@gmail.com	TypeScript	1	0
4	9b0a373ffce52322fd995f7f43ce74be6e389941	canalcode	2025-07-09 00:00:00	henriqueduran15@gmail.com	TypeScript	37	0
5	9b0a373ffce52322fd995f7f43ce74be6e389941	canalcode	2025-07-09 00:00:00	henriqueduran15@gmail.com	TypeScript	221	0
6	9b0a373ffce52322fd995f7f43ce74be6e389941	canalcode	2025-07-09 00:00:00	henriqueduran15@gmail.com	TypeScript	28	0
7	9b0a373ffce52322fd995f7f43ce74be6e389941	canalcode	2025-07-09 00:00:00	henriqueduran15@gmail.com	TypeScript	6	0
8	3e30b9b8aec3d62b6c8c54f72187732f9bfd2593	canalcode	2025-07-09 00:00:00	joaopedrozrnm@usp.br	TypeScript	5	1
9	3e30b9b8aec3d62b6c8c54f72187732f9bfd2593	canalcode	2025-07-09 00:00:00	joaopedrozrnm@usp.br	TypeScript	2	2
10	3e30b9b8aec3d62b6c8c54f72187732f9bfd2593	canalcode	2025-07-09 00:00:00	joaopedrozrnm@usp.br	TypeScript	19	1
11	3e30b9b8aec3d62b6c8c54f72187732f9bfd2593	canalcode	2025-07-09 00:00:00	joaopedrozrnm@usp.br	TypeScript	36	0
12	3e30b9b8aec3d62b6c8c54f72187732f9bfd2593	canalcode	2025-07-09 00:00:00	joaopedrozrnm@usp.br	TypeScript	6	4
13	3e30b9b8aec3d62b6c8c54f72187732f9bfd2593	canalcode	2025-07-09 00:00:00	joaopedrozrnm@usp.br	TypeScript	6	5
14	3e30b9b8aec3d62b6c8c54f72187732f9bfd2593	canalcode	2025-07-09 00:00:00	joaopedrozrnm@usp.br	TypeScript	1	1
15	3e30b9b8aec3d62b6c8c54f72187732f9bfd2593	canalcode	2025-07-09 00:00:00	joaopedrozrnm@usp.br	TypeScript	2	2
16	3e30b9b8aec3d62b6c8c54f72187732f9bfd2593	canalcode	2025-07-09 00:00:00	joaopedrozrnm@usp.br	TypeScript	2	2
17	9fa788a37bb2c3b09e21cb55faf3e50321a73767	canalcode	2025-07-10 00:00:00	henriqueduran15@gmail.com	TypeScript	7	0
18	9fa788a37bb2c3b09e21cb55faf3e50321a73767	canalcode	2025-07-10 00:00:00	henriqueduran15@gmail.com	TypeScript	1	0
19	9fa788a37bb2c3b09e21cb55faf3e50321a73767	canalcode	2025-07-10 00:00:00	henriqueduran15@gmail.com	TypeScript	37	0
20	9fa788a37bb2c3b09e21cb55faf3e50321a73767	canalcode	2025-07-10 00:00:00	henriqueduran15@gmail.com	TypeScript	221	0
21	9fa788a37bb2c3b09e21cb55faf3e50321a73767	canalcode	2025-07-10 00:00:00	henriqueduran15@gmail.com	TypeScript	28	0
22	9fa788a37bb2c3b09e21cb55faf3e50321a73767	canalcode	2025-07-10 00:00:00	henriqueduran15@gmail.com	TypeScript	6	0
23	97febdbba2f6b434f67993d4d4a4c0243f1d5ae3	canalcode	2025-07-10 00:00:00	henriqueduran15@gmail.com	TypeScript	5	1
24	97febdbba2f6b434f67993d4d4a4c0243f1d5ae3	canalcode	2025-07-10 00:00:00	henriqueduran15@gmail.com	TypeScript	2	2
25	97febdbba2f6b434f67993d4d4a4c0243f1d5ae3	canalcode	2025-07-10 00:00:00	henriqueduran15@gmail.com	TypeScript	19	1
26	97febdbba2f6b434f67993d4d4a4c0243f1d5ae3	canalcode	2025-07-10 00:00:00	henriqueduran15@gmail.com	TypeScript	36	0
27	97febdbba2f6b434f67993d4d4a4c0243f1d5ae3	canalcode	2025-07-10 00:00:00	henriqueduran15@gmail.com	TypeScript	6	4
28	97febdbba2f6b434f67993d4d4a4c0243f1d5ae3	canalcode	2025-07-10 00:00:00	henriqueduran15@gmail.com	TypeScript	6	5
29	97febdbba2f6b434f67993d4d4a4c0243f1d5ae3	canalcode	2025-07-10 00:00:00	henriqueduran15@gmail.com	TypeScript	1	1
30	97febdbba2f6b434f67993d4d4a4c0243f1d5ae3	canalcode	2025-07-10 00:00:00	henriqueduran15@gmail.com	TypeScript	2	2
31	97febdbba2f6b434f67993d4d4a4c0243f1d5ae3	canalcode	2025-07-10 00:00:00	henriqueduran15@gmail.com	TypeScript	2	2
32	2cb3f4f3773d6f887413f0aef86d05bed66903b4	canalcode	2025-07-26 00:00:00	joaopedrozrnm@usp.br	TypeScript	212	130
33	2cb3f4f3773d6f887413f0aef86d05bed66903b4	canalcode	2025-07-26 00:00:00	joaopedrozrnm@usp.br	TypeScript	4	0
34	2cb3f4f3773d6f887413f0aef86d05bed66903b4	canalcode	2025-07-26 00:00:00	joaopedrozrnm@usp.br	Python	4	0

Figura 7 – Exemplo de saída do *script*.

Ressalta-se que, no resultado do *script*, cada linha corresponde não a um *commit*, mas a um arquivo modificado de um *commit*, por isso existem várias linhas da planilha com o mesmo valor na coluna *hash*, como pode ser observado na Figura 7, acima.

Após a geração, o arquivo pode ser enviado ao sistema por meio da interface de *upload*.

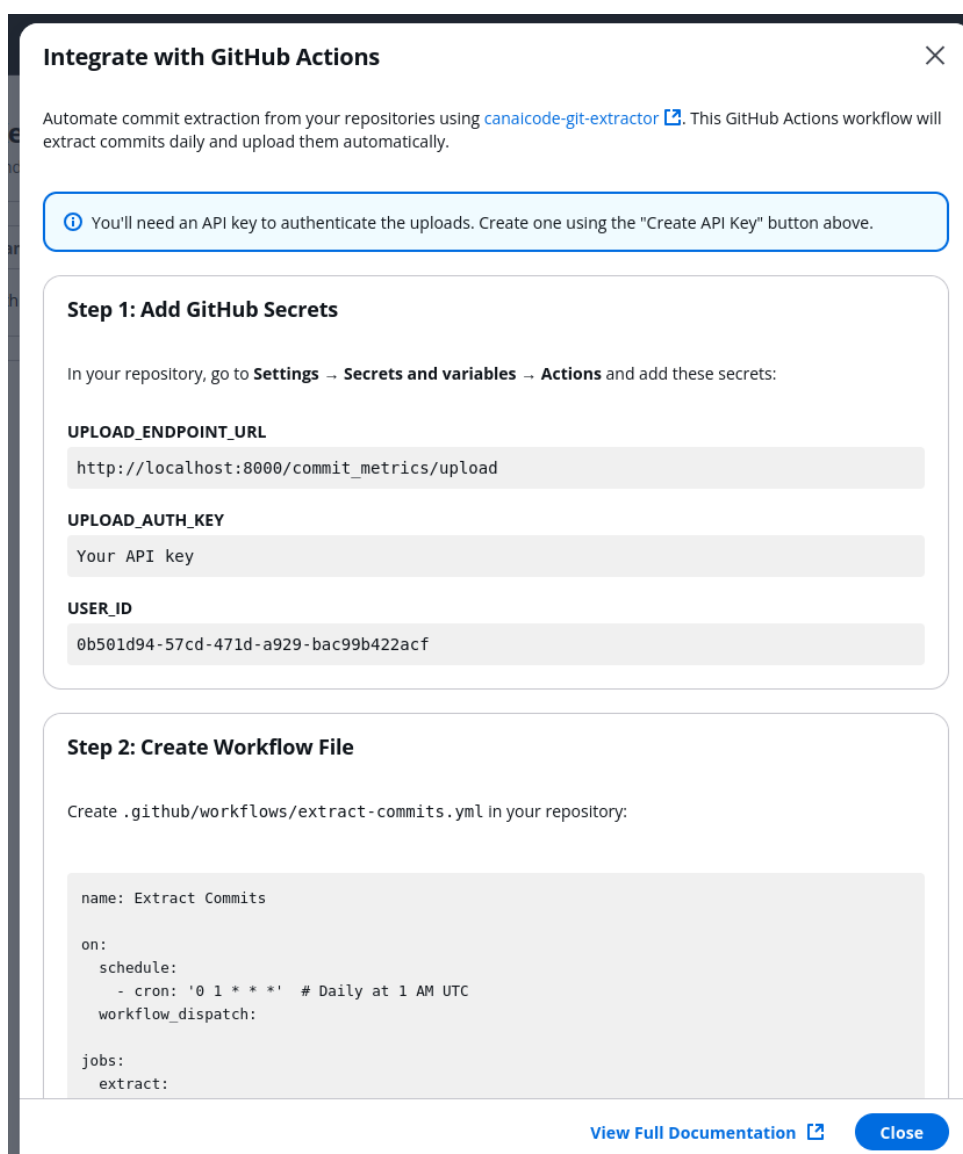
(2) Forma automática.

A fim de dar mais comodidade ao usuário deste sistema, foi desenvolvido uma forma de automatizar a coleta de dados de *commits* do repositório de código. Desta forma, além de poder fazer o *upload* manual dos dados de *commit* a partir de um arquivo *excel*, como explicado anteriormente, o sistema oferece a capacidade de integrar-se com o repositório *GitHub* do usuário por meio da ferramenta *Github Actions*.

Esta ferramenta permite que *scripts* sejam rodados em uma máquina provisionada

pelo próprio *GitHub* mediante ações pré-configuradas. Ações típicas que ativam esta ferramenta são intervalo de tempo ou novos *commits* em determinadas *branches* do repositório. No caso do *script* que será executado para a extração de *commits*, a configuração padrão é de que seja executado diariamente.

No caso deste sistema, foi desenvolvido um *script* específico que, além de coletar os dados dos *commits*, efetua o *upload* para a rota do backend. Entretanto, para que o *script* seja capaz de autenticar-se no sistema para efetuar o *upload* dos dados, foi desenvolvida uma funcionalidade de autenticação por meio de chaves de *API*, visto que não é seguro que o usuário configure sua própria senha em um *script* automatizado. A tela de configuração mencionada é ilustrada nas figuras 8 e 9.



Integrate with GitHub Actions ✕

Automate commit extraction from your repositories using [canalcode-git-extractor](#). This GitHub Actions workflow will extract commits daily and upload them automatically.

📘 You'll need an API key to authenticate the uploads. Create one using the "Create API Key" button above.

Step 1: Add GitHub Secrets

In your repository, go to **Settings** → **Secrets and variables** → **Actions** and add these secrets:

UPLOAD_ENDPOINT_URL
http://localhost:8000/commit_metrics/upload

UPLOAD_AUTH_KEY
Your API key

USER_ID
0b501d94-57cd-471d-a929-bac99b422acf

Step 2: Create Workflow File

Create `.github/workflows/extract-commits.yml` in your repository:

```
name: Extract Commits

on:
  schedule:
    - cron: '0 1 * * *' # Daily at 1 AM UTC
  workflow_dispatch:

jobs:
  extract:
```

[View Full Documentation](#) [Close](#)

Figura 8 – Guia de configuração do GithubActions - Parte 1

Desta forma, criou-se uma tela para que o usuário consiga criar e revogar chaves de *API*. Este fluxo segue os padrões comerciais de segurança na autenticação por chaves de

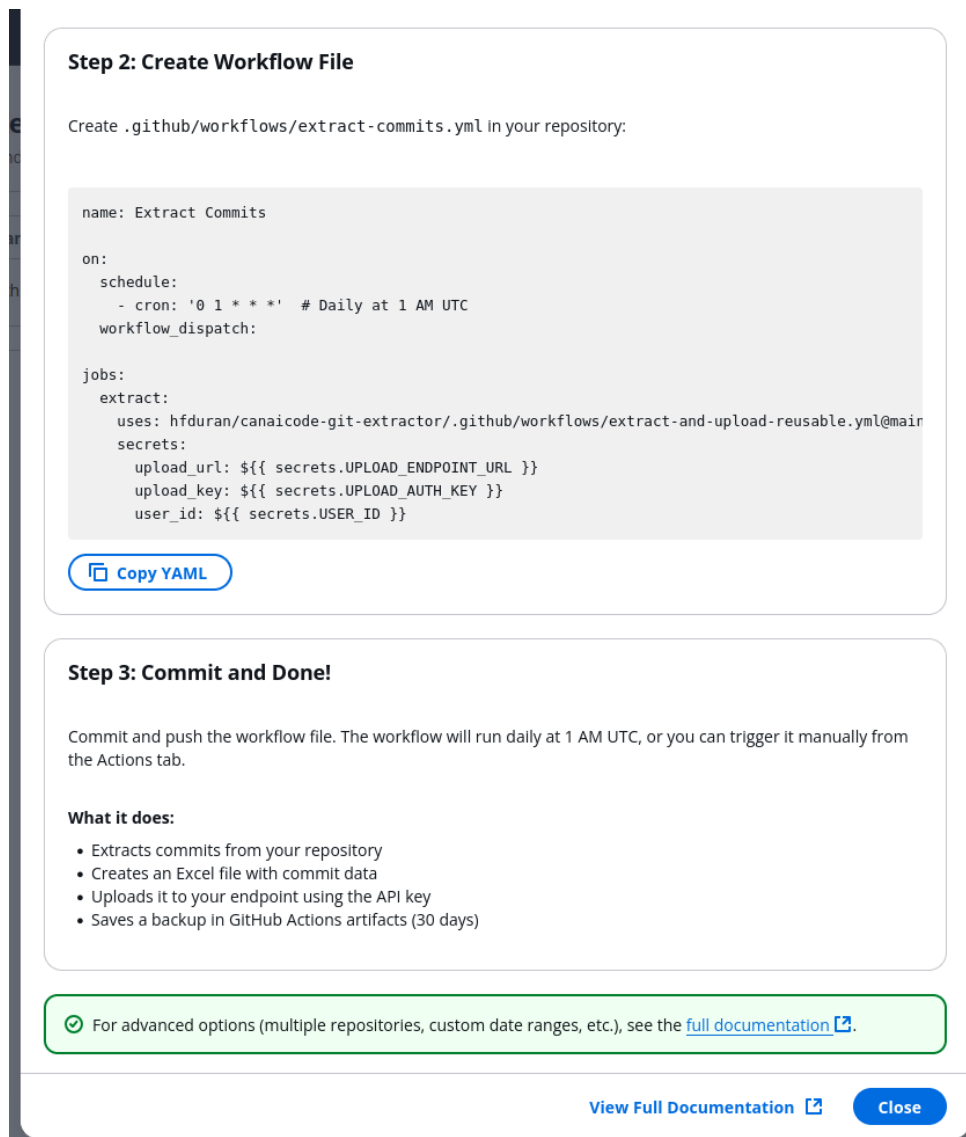


Figura 9 – Guia de configuração do GithubActions - Parte 2

API, pois, após a criação da chave, ela nunca mais é retornada por completo ao usuário. O usuário tem uma única oportunidade de copiar a chave e guardá-la em local seguro. A fim de que a experiência seja mais cômoda, o sistema oferece a possibilidade do usuário criar apelidos para cada chave. Além disso, as chaves fornecem acesso limitado ao sistema, permitindo autenticação apenas à rota de *upload* de dados. A Figura 11 demonstra o gerenciamento das *API Keys*.

Por conseguinte, na própria tela de gerenciamento de chaves de *API*, o usuário é apresentado com um guia detalhado de como configurar a ferramenta de *Github Actions*, com as três variáveis que exigem configuração: (i) identificador de usuário, (ii) *URL* do backend e (iii) a própria chave de *API*.

O código deste *script* é aberto, podendo ser conferido pelo próprio usuário ao clicar em *View Full Documentation*, que o levará a [este repositório](#).

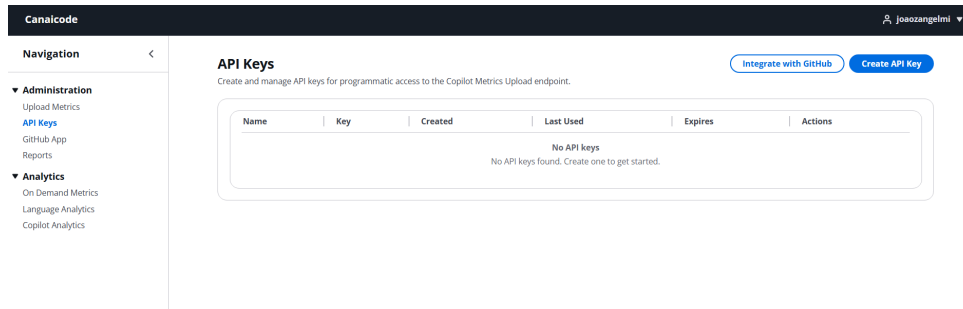


Figura 10 – Tela de cadastro das chaves de API

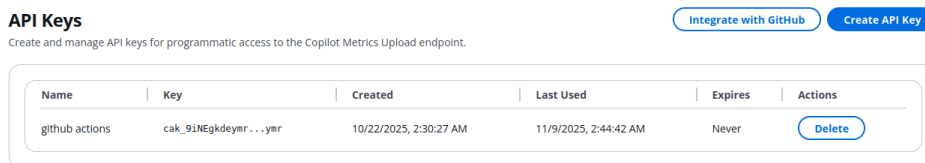
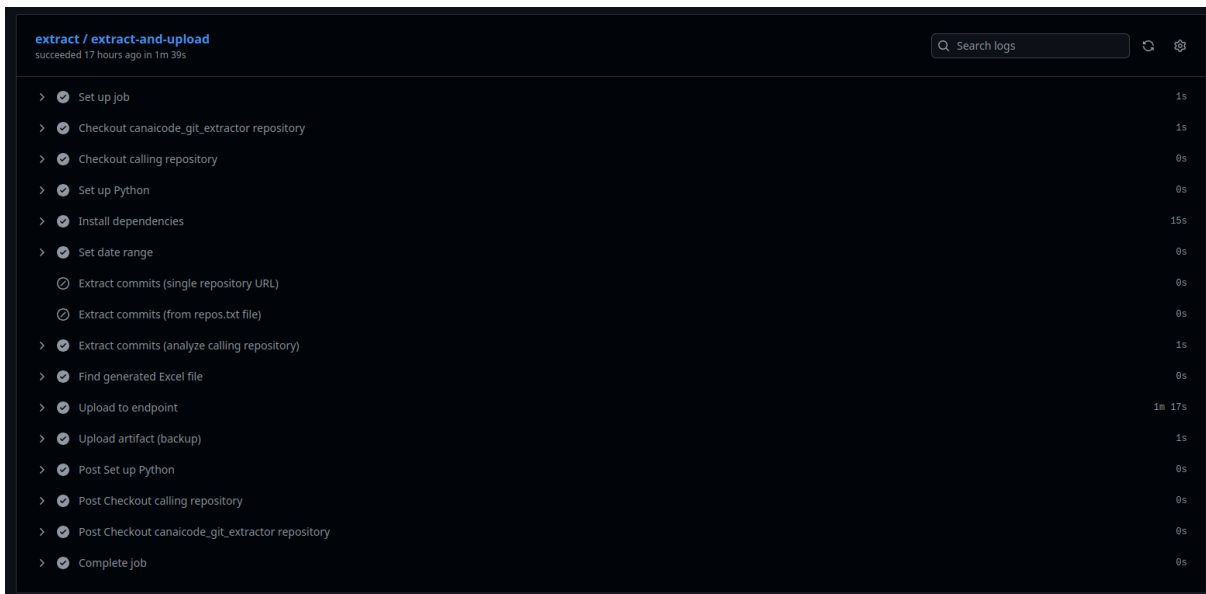


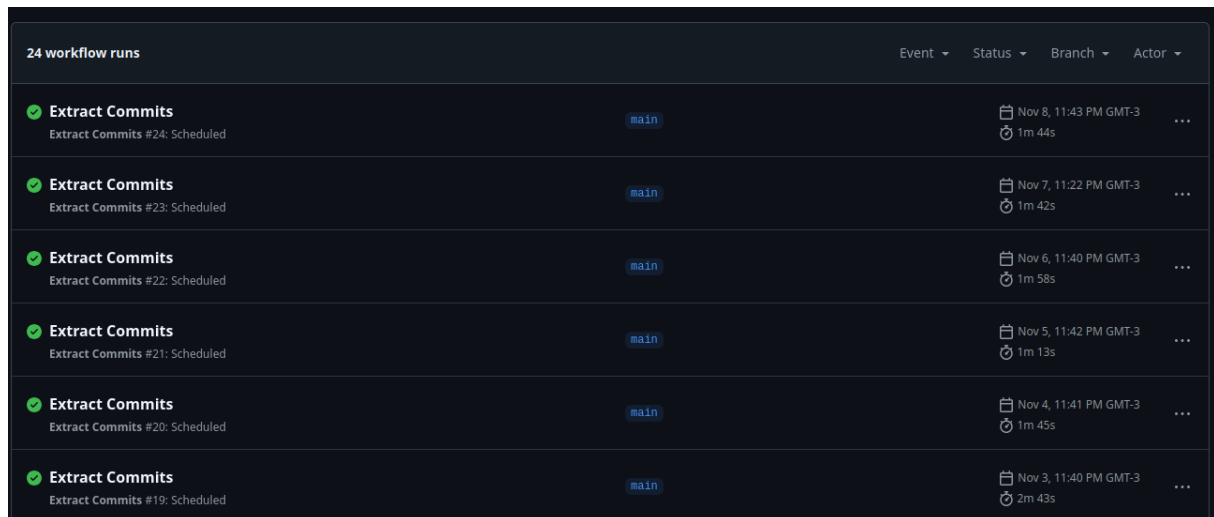
Figura 11 – Tela de gerenciamento de chaves de API

Na Figura 12 é ilustrada uma rotina de execução manual do *script*.

Figura 12 – Exemplo de execução do *script* no *GithubActions*

Por último, é exemplificada a execução, de forma automática, do *script* na Figura 13.

Em ambas as modalidades, os dados coletados são armazenados no banco de dados em seu formato bruto, sem a aplicação imediata de transformações ou cálculos de métricas derivadas. Desta forma, as métricas são calculadas dinamicamente conforme os parâmetros definidos pelo usuário, durante a etapa de análise, apresentada nas seções subsequentes.



The screenshot shows a list of 24 workflow runs for the 'Extract Commits' workflow. The table has columns for Event, Status, Branch, and Actor. The runs are scheduled and occur daily at 11:43 PM GMT-3. Each run is associated with a specific commit hash and a duration (e.g., 1m 44s, 1m 42s, 1m 58s, 1m 13s, 1m 45s, 2m 43s).

Event	Status	Branch	Actor
Extract Commits #24: Scheduled	main	Nov 8, 11:43 PM GMT-3	1m 44s
Extract Commits #23: Scheduled	main	Nov 7, 11:22 PM GMT-3	1m 42s
Extract Commits #22: Scheduled	main	Nov 6, 11:40 PM GMT-3	1m 58s
Extract Commits #21: Scheduled	main	Nov 5, 11:42 PM GMT-3	1m 13s
Extract Commits #20: Scheduled	main	Nov 4, 11:41 PM GMT-3	1m 45s
Extract Commits #19: Scheduled	main	Nov 3, 11:40 PM GMT-3	2m 43s

Figura 13 – Exemplo de execuções diárias do *script* no GithubActions

Além disso, o sistema foi projetado para garantir que não ocorram duplicações de registros. Durante o processo de inserção, é realizada uma verificação de integridade que impede a regravação de *commits* previamente armazenados, ainda que o mesmo conjunto de dados seja submetido novamente via *upload*.

A não duplicação dos dados é feita a partir de índices no próprio banco de dados. Esta é uma forma segura de garantir integridade de dados, pois é certificado pelo próprio gerenciador de banco de dados, não pela aplicação.

Os índices de unicidade escolhidos para o armazenamento de *commits* são as informações de *hash*, nome de repositório e linguagem de programação. Isso porque, em cada *commit*, são extraídas apenas uma única informação de linguagem de programação para cada *hash* de *commit*. O nome de repositório é necessário, pois um usuário poderia ter dois repositórios idênticos com nomes diferentes. Desta forma, o *hash* do *commit* não pode ser tratado como único para cada repositório no sistema.

A implementação desse índice de unicidade pode ser vista no trecho de código presente na Figura 14.

Com os índices disponíveis, a estratégia de armazenamento de dados adotada é a de "*Easier to Ask Forgiveness Than Permission*", ou seja, tentar salvar os dados sem verificar a existência deles no banco de dados por parte da aplicação. O programa tenta salvar os dados, caso um erro de conflito for erguido, ignora-se. Isso é porque assume-se que, se os índices existentes apontaram conflito, há uma tentativa de armazenar duas vezes o mesmo dado, tornando seguro ignorar o erro. Hunner (2020) discute ambas abordagens: *Easier to Ask Forgiveness Than Permission* (EAFP) e *Look Before You Leap* (LBYL) representam dois padrões distintos de coordenação e tratamento de erros. Enquanto LBYL envolve

```
1 class RawCommitMetrics(Base):
2     __tablename__ = "raw_commit_metrics"
3
4     id = Column(String, primary_key=True, index=True)
5     hash = Column(String)
6     repository_name = Column(String)
7     repository_team = Column(String, index=True)
8     date = Column(DateTime, index=True)
9     author_name = Column(String)
10    author_teams = Column(String, index=True)
11    language = Column(String)
12    added_lines = Column(Integer)
13    removed_lines = Column(Integer)
14    created_at = Column(DateTime, default=lambda: datetime.now(
15        timezone.utc))
16    user_id = Column(String, index=True)
17
18    __table_args__ = (
19        UniqueConstraint('hash', 'repository_name', 'language', name
20            = 'unique_commit_per_repo_lang'),
21    )
```

Figura 14 – Definição da entidade de Métricas de Commit, apresentando índices de unicidade

verificar previamente se a operação é segura — adicionando etapas extras de leitura e abrindo espaço para condições de corrida — a abordagem EAFP tenta executar diretamente a ação desejada e trata apenas os conflitos efetivamente ocorridos. Em sistemas com alta concorrência ou inserções frequentes, EAFP tende a ser mais performática e simples de implementar, uma vez que elimina verificações redundantes e delega ao mecanismo de exceções ou ao banco de dados a resolução de colisões.

Assim, a estratégia tomada é consideravelmente mais eficiente que a alternativa de LBYL, que iria requerer que a aplicação checasse primeiramente se o dado existe na base de dados, para então executar a função de armazenamento.

A estrutura completa da entidade é apresentada na Tabela 2, a seguir.

Tabela 2 – Estrutura das Métricas de *Commits*

Coluna	Tipo de Dado	Legenda
id	text	Identificador único
hash	text	Identificador do <i>commit</i>
repository_name	text	Nome do repositório no <i>GitHub</i>
repository_team	text	Nome do time associado ao repositório no <i>GitHub</i> (opcional)
date	datetime	Data do evento
author_name	text	Nome de usuário ou email do autor do <i>commit</i>
author_teams	text	Times associados ao autor do <i>commit</i> no <i>GitHub</i> (opcional)
language	text	Linguagem de programação
added_lines	integer	Total de linhas adicionadas
removed_lines	integer	Total de linhas removidas
created_at	datetime	Data de inserção da métrica
user_id	text	Identificador do usuário associado

5.3.1.2 Métricas do *GitHub Copilot*

As métricas de uso do *GitHub Copilot* constituem o segundo conjunto de dados empregados neste estudo. Esses dados são disponibilizados via *API* oficial do *GitHub*, em formato *JSON*.

O arquivo *JSON* contém uma lista de objetos, cada um representando um conjunto agregado de métricas para uma determinada data e ambiente de uso. Essas informações incluem, entre outros campos, o número de usuários ativos, o total de interações registradas, o editor de código (*IDE*) utilizado, o modelo do *Copilot* empregado e os eventos de cópia e inserção ocorridos durante o uso da ferramenta.

A etapa de transformação e padronização desses dados é executada pela classe *GhCopilotConsumer*, responsável por processar o conteúdo bruto do arquivo *JSON* submetido e convertê-lo em estruturas de dados compatíveis com o banco de dados do sistema. O método central dessa classe, denominado `get_metrics`: realiza a leitura, validação e mapeamento dos registros em duas categorias principais - métricas de código e métricas de conversação. O trecho de código apresentado na Figura 15 ilustra parte desse processo:

```

1 for entry in data:
2     parsed_entry = CopilotMetricsEntry.model_validate(entry)
3     if(parsed_entry.copilot_id_code_completions.total_engaged_users
4         > 0):
5         for editor in parsed_entry.copilot_id_code_completions.
6             editors:
7             for model in editor.models:
8                 for language in model.languages:
9                     result["code"].append(
10                         CopilotCodeMetrics(
11                             id=str(uuid.uuid4()),
12                             code_acceptances=language.
13                                 total_code_acceptances,
14                             code_suggestions=language.
15                                 total_code_suggestions,
16                             copilot_model=model.name,
17                             date=parsed_entry.date,
18                             IDE=editor.name,
19                             language=language.name,
20                             lines_accepted=language.
21                                 total_code_lines_accepted,
22                             lines_suggested=language.
23                                 total_code_lines_suggested,
24                             total_users=language.total_engaged_users
25                             ,
26                             user_id=user_id
27                         )
28                     )
29             )
30     )

```

Figura 15 – Trecho do método `get_metrics` responsável pela extração e transformação dos dados do *GitHub Copilot*

O método percorre de forma hierárquica os elementos de cada entrada e extrai as informações para análise. Para cada combinação identificada, é criado um objeto das classes `CopilotCodeMetrics` ou `CopilotChatMetrics`, conforme o tipo de métrica. Esses objetos são populados com os respectivos dados. Cada registro é identificado de forma única por meio de um *UUID* gerado e também é associado ao usuário responsável pelo carregamento do arquivo.

O resultado final do método é uma estrutura de dicionário contendo duas listas — uma para métricas de código e outra para métricas de *chat*.

Após a transformação, as informações são armazenadas no banco de dados, preservando a granularidade original das informações. As Tabelas 3 e 4 apresentam a estrutura final esperada para os registros de métricas do *Copilot*.

Assim como é feito para os *commits*, é necessário assegurar de que não haverá duplicação no banco, para os dados do *Copilot*. Para tanto, são utilizados índices de unicidade, baseados nas seguintes informações: (i) nome de time, (ii) data, (iii) IDE, e (iv) modelo. Verificou-se que tais informações podem ser consideradas únicas e garantem consistência dos dados.

Tabela 3 – Estrutura das Métricas de Uso do Assistente do *GitHub Copilot*

Coluna	Tipo de Dado	Legenda
id	text	Identificador único
team_name	text	Nome do time no <i>GitHub</i> (opcional)
date	datetime	Data do evento
ide	text	IDE
copilot_model	text	Modelo do <i>GitHub Copilot</i>
language	text	Linguagem de programação
total_users	integer	Total de usuários do assistente de código com essa linguagem, essa IDE e esse modelo
code_suggestions	integer	Total de sugestões de código
lines_accepted	integer	Total de linhas de código das sugestões aceitas
lines_suggested	integer	Total de linhas de código sugeridas
created_at	datetime	Data de inserção da métrica
user_id	text	Identificador do usuário associado

Tabela 4 – Estrutura das Métricas de Uso do *Chat* do *GitHub Copilot*

Coluna	Tipo de Dado	Legenda
id	text	Identificador único
team_name	text	Nome do time no <i>GitHub</i> (opcional)
date	datetime	Data do evento
ide	text	IDE
copilot_model	text	Modelo do <i>GitHub Copilot</i>
total_users	integer	Total de usuários do <i>chat</i> nessa IDE e modelo
total_chats	integer	Total de mensagens enviadas para o <i>chat</i>
copy_events	integer	Total de cópias de respostas do <i>Copilot</i>
insertion_events	integer	Total de eventos de inserção
created_at	datetime	Data de inserção da métrica
user_id	text	Identificador do usuário associado

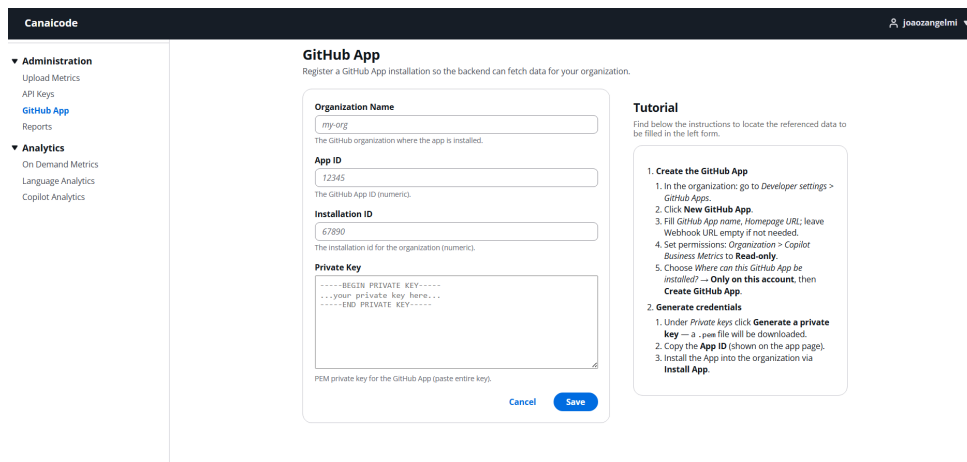
As métricas do *Copilot*, assim como as métricas de *commits*, podem ser adicionadas ao sistema de duas maneiras: com o gestor fazendo a extração manualmente e submetendo o arquivo por meio da interface de *upload* ou com o gestor configurando uma permissão para o sistema fazer o acesso a essa *API* oficial e extrair os dados automaticamente.

(1) Forma manual.

No caso do usuário optar pela modalidade manual, a responsabilidade pela obtenção do arquivo *JSON* contendo as métricas de uso do *Copilot* é delegada ao próprio usuário. Após a extração dos dados por meios externos ao sistema desenvolvido, o usuário realiza o carregamento do arquivo por meio da interface de *upload* de métricas do sistema (Figura 5). A plataforma, subsequentemente, inicia o processamento dos dados contidos no arquivo submetido.

(2) Forma automática.

Na modalidade automática, o sistema realiza a coleta diretamente a partir da *API* oficial do *GitHub Copilot*. Nessa abordagem, o usuário cria um *GitHub App* com autorização de acesso somente para leitura das métricas do *Copilot*, associa à sua organização e fornece suas credenciais de integração, como ilustrado na Figura 16. A aplicação utiliza a *API* oficial do GitHub para coletar o *JSON* diariamente, realizando chamadas autenticadas e armazenando as informações obtidas na mesma estrutura utilizada para os dados provenientes do método manual.



The screenshot shows the 'GitHub App' registration page within the 'Canalcode' application. The interface has a dark header with the 'Canalcode' logo and a user profile 'joaozangelmi'. A sidebar on the left contains navigation links under 'Administration' (Upload Metrics, API Keys, GitHub App, Reports) and 'Analytics' (On Demand Metrics, Language Analytics, Copilot Analytics). The main content area is titled 'GitHub App' and includes the instruction: 'Register a GitHub App installation so the backend can fetch data for your organization.' The form contains four fields: 'Organization Name' (with 'my-org' entered), 'App ID' (with '12345' entered), 'Installation ID' (with '67890' entered), and 'Private Key' (a large text area with a placeholder for a PEM private key). Below the form are 'Cancel' and 'Save' buttons. To the right of the form is a 'Tutorial' section with two numbered steps: '1. Create the GitHub App' and '2. Generate credentials', each with sub-instructions.

Figura 16 – Tela de cadastro do *GitHub App*.

Como os dados de repositórios de uma organização são sensíveis, a automação de obtenção dos dados de uso da IA foi projetada para que nenhum acesso a repositórios seja necessário nessa etapa. Na configuração do *GitHub App*, conforme ilustrado na Figura 16, o usuário precisa conceder apenas a permissão de leitura das métricas do *GitHub Copilot*, sem qualquer autorização relacionada a repositórios ou outros dados internos.

Após a criação do *App* e sua associação à organização, o usuário deve preencher o formulário com o nome da organização, o *App ID*, o *Installation ID* e a *Private key*. Por se tratar de um dado sensível, a chave privada é armazenada com uma camada de criptografia, utilizando uma solução simétrica por meio da ferramenta *Fernet* da biblioteca *Cryptography*. Todos esses dados são salvos no banco de dados, representada na Tabela 5, que é acessada pelo *worker* diariamente para autenticação na *API* de obtenção das métricas.

Para obtenção automática das métricas, o *worker* é executado diariamente por meio de *schedulers*. É feita a busca de todos os *GitHub Apps* salvos no banco de dados e em seguida o sistema itera por cada um deles, autenticando-se, buscando os dados das métricas do *Copilot* e utilizando da mesma etapa de padronização e transformação de dados detalhada no início desta subseção.

O programa de *scheduler* fica hospedado na mesma máquina que o backend do website. No processo de *deploy*, além do comando de inicializar a *API*, é também configurado

Tabela 5 – Estrutura do Cadastro de *GitHub Apps*

Coluna	Tipo de Dado	Legenda
id	text	Identificador único
organization_name	text	Nome da organização no <i>GitHub</i>
app_id	text	Identificador <i>GitHub App</i>
installation_id	text	Identificador da instalação do <i>GitHub App</i> na organização
private_key_encrypted	text	Chave privada criptografada
created_at	datetime	Data de cadastro do <i>GitHub App</i>
user_id	text	Identificador do usuário associado

um comando que execute o *scheduler*, efetivamente criando dois processos distintos.

5.3.2 Tratamento de dados e cálculo de métricas

Com os dados de *commits* e de uso do *GitHub Copilot* devidamente armazenados, inicia-se a etapa de tratamento e cálculo das métricas. São aplicados procedimentos de limpeza, normalização e cruzamento das informações, assegurando que os conjuntos de dados estejam compatíveis para análise.

5.3.2.1 *Productivity Analytics* - Frontend

Diferentemente das demais telas, essa seção não apresenta um gráfico inicial. O usuário deve primeiro configurar um conjunto de parâmetros que determinam a forma como as métricas serão calculadas e exibidas. Essa configuração permite que as análises sejam geradas sob demanda, de acordo com o contexto e o nível de detalhamento desejado. Os parâmetros disponíveis aos usuários são:

- Período de agregação dos dados — define o intervalo temporal de agrupamento das métricas, podendo ser semanal, mensal, trimestral ou anual;
- Tipo de métrica — permite alternar entre métricas baseadas em linhas de código e em *commits*;
- Time de desenvolvimento — parâmetro que filtra os dados por equipes cadastradas no GitHub;
- Linguagem de programação — filtro opcional que permite restringir a análise a n linguagens específicas;
- Data inicial e data final — delimitam o período de análise.

A tela de configuração dos parâmetros, descritos acima, é disposta abaixo, na Figura 17.

Figura 17 – Tela de configuração dos parâmetros para tela de *Productivity Analytics*.

Após a configuração dos parâmetros e o envio da requisição, o sistema realiza o processamento dos dados no backend, agrupando e correlacionando as métricas solicitadas em tempo de execução.

Quando o usuário seleciona o tipo de métrica baseado em linhas de código, o sistema apresenta, para o período e agregação definidos, um gráfico de barras contendo duas informações principais: a quantidade de linhas modificadas sem o uso do *Copilot* (em azul) e a quantidade de linhas modificadas com auxílio da ferramenta (em verde).

Além da visualização gráfica, ao posicionar o cursor sobre as barras de um período específico, são exibidos os seguintes valores absolutos e relativos:

1. o total de linhas adicionadas;
2. o total de linhas adicionadas com auxílio do *Copilot*; e
3. a porcentagem de contribuição do *Copilot* no total produzido.

O que permite uma análise detalhada da participação do *Copilot* no volume de código produzido.

A Figura 18, abaixo, mostra o gráfico descrito.

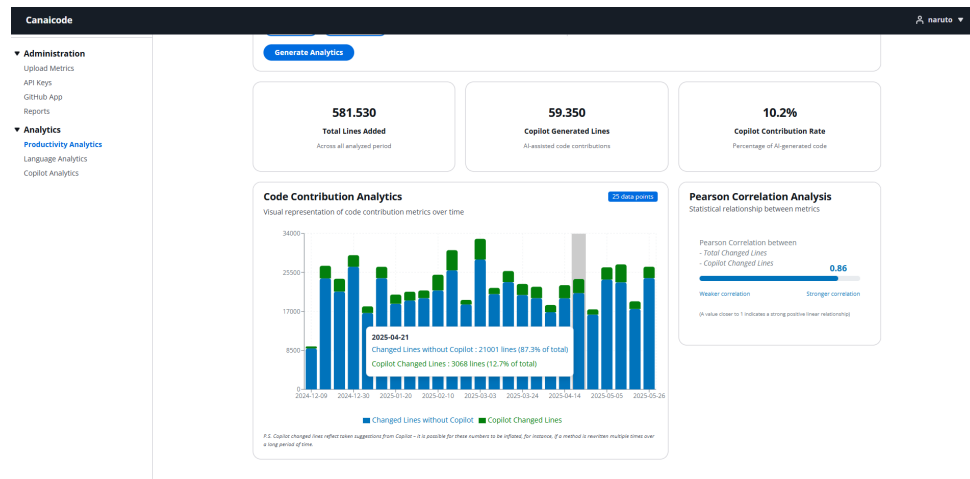


Figura 18 – Gráfico de produtividade por linhas de código.

Complementarmente, é exibido o valor da correlação de *Pearson* entre o total de linhas adicionadas e o total de linhas adicionadas pelo *Copilot*, também aparente na Figura 18.

Quando o tipo de métrica selecionado é baseado em *commits*, a tela exibe o total de *commits* realizados em cada período definido, considerando os mesmos filtros disponíveis.

5.3.2.2 Productivity Analytics - Backend

A integração entre as métricas de *commits* e as métricas de uso do *GitHub Copilot* constitui uma etapa central do processo analítico do sistema, pois é o que permite relacionar ações de codificação realizadas nos repositórios *Git* ao uso efetivo da ferramenta de inteligência artificial. Embora esses dois conjuntos de dados sejam coletados e armazenados de forma independente, suas estruturas são definidas de modo compatível, possibilitando o alinhamento por meio de atributos comuns: data, linguagem de programação e autor.

A lógica responsável por esse processo encontra-se encapsulada na classe `GetCalculatedMetricsUseCase`. Essa unidade da aplicação tem a função de consolidar os dados brutos provenientes dos repositórios de *commits* e das métricas de uso do *Copilot*, realizando a filtragem, o alinhamento e a agregação necessários para produzir as métricas finais utilizadas nos *dashboards*. Quando o usuário realiza uma requisição a partir da interface, os parâmetros configurados (como o período de análise, o tipo de métrica desejada, o time de desenvolvimento e a linguagem de programação) são recebidos pela API e encaminhados ao serviço de cálculo de métricas de produtividade. Esse serviço atua como uma camada intermediária entre os repositórios de dados e as estruturas de resposta retornadas ao *frontend*, garantindo que todo o processamento seja realizado de forma coerente com os filtros definidos pelo usuário.

O ponto de entrada para essa lógica é o método `execute`, que recebe como parâ-

metros: identificador do usuário, intervalo temporal selecionado, métrica de produtividade desejada, e conjunto de linguagens filtradas. O método principal dessa funcionalidade, ilustrado na Figura 19, inicia buscando os dados brutos de *commits* e de código gerado com auxílio do *Copilot*, por meio dos repositórios *commit_metrics_repository* e *copilot_code_metrics_repository*. Esses repositórios retornam listas de objetos representando as métricas armazenadas no banco de dados, já filtradas conforme os parâmetros informados pelo usuário, como intervalo de datas e linguagens. O trecho de código a seguir apresenta a estrutura desse método orquestrador, destacando as etapas iniciais de carregamento e filtragem dos dados, e indicando sua responsabilidade em coordenar os demais métodos da classe para a conclusão do processamento.

```
1 def execute(  
2     self,  
3     user_id: str,  
4     period: Period,  
5     productivity_metric: Productivity_metric,  
6     initial_date: Optional[datetime] = None,  
7     final_date: Optional[datetime] = None,  
8     languages: Optional[List[str]] = None,  
9 ) -> CalculatedMetrics:  
10     raw_commit_metrics = self.commit_metrics_repository.listByUserId(  
11         user_id, initial_date, final_date, languages  
12     )  
13  
14     if not raw_commit_metrics:  
15         return CalculatedMetrics(  
16             user_id=user_id,  
17             languages=[],  
18             period=period,  
19             data=[],  
20         )  
21  
22     raw_copilot_code_metrics = self.copilot_code_metrics_repository.  
23         listByUserId(  
24             user_id, initial_date, final_date, languages  
25         )  
26  
27     productivity_metric_map = {  
28         Productivity_metric.code_lines: self.get_code_lines_metrics,  
29         Productivity_metric.commits: self.get_commit_metrics,  
30     }  
31  
32     result = productivity_metric_map[productivity_metric](  
33         raw_commit_metrics, raw_copilot_code_metrics, period  
34     )  
35     return result
```

Figura 19 – Método `execute` da classe `GetCalculatedMetricsUseCase`

A função `listByUserId` aplicada a ambos os repositórios assegura que os dados

retornados pertencem exclusivamente ao usuário autenticado, respeitando as restrições de acesso. Além disso, os dois conjuntos já são filtrados, de forma que compartilham necessariamente três propriedades fundamentais: (i) o identificador do usuário (`user_id`); (ii) o intervalo temporal considerado (`date`); e (iii) o conjunto de linguagens filtradas (`language`). Esses atributos permitem o alinhamento entre os conjuntos, de forma que os dados analisados sejam compatíveis quando forem cruzados.

Uma vez carregados os dados brutos, o método seleciona a métrica de produtividade a ser calculada. Esse processo ocorre por meio de um mapa interno que relaciona o tipo de métrica solicitada (`code_lines` ou `commits`) à função de agregação apropriada, conforme também é explicitado no trecho `??`, em `productivity_metric_map`.

As funções `get_code_lines_metrics` e `get_commit_metrics` são responsáveis pelo cruzamento efetivo entre os dois conjuntos de dados. Uma vez que os dados já foram filtrados por usuário e por parâmetros de consulta, ambas seguem o mesmo princípio metodológico: agregam valores por data e linguagem, organizam os registros em séries temporais coerentes e, ao final, realizam a fusão dos agrupamentos de `commits` com aqueles provenientes do uso do *Copilot*.

Quando o tipo de métrica selecionado pelo usuário é o de linhas de código, o sistema aciona o método dedicado. Inicialmente, os dados brutos extraídos dos repositórios de `commits` e de uso do *Copilot* são convertidos em estruturas tabulares utilizando a biblioteca `pandas`. Essa conversão possibilita a aplicação de operações de agrupamento temporal segundo a periodicidade especificada pelo usuário (diária, semanal, mensal ou outro intervalo permitido). A agregação é realizada por meio do recurso `pd.Grouper`, que organiza os registros de ambas as fontes.

O trecho de código representado na Figura 20 ilustra essa etapa inicial de preparação e agrupamento:

Após o agrupamento temporal, a função percorre cada grupo de `commits`, calculando a quantidade líquida de linhas modificadas naquele intervalo. Durante essa iteração, são identificadas as linguagens utilizadas e os autores que contribuíram para aquele período, e esses dados são adicionados progressivamente à instância de `CalculatedMetrics`, responsável por consolidar as métricas agregadas de produtividade.

Na segunda etapa, o método realiza o cruzamento das métricas calculadas com os dados provenientes do *Copilot*. Para cada intervalo temporal, soma-se o total de linhas adicionadas com auxílio da ferramenta utilizando o método `calculate_gross_use_of_AI_lines`. Em seguida, é calculada a porcentagem relativa entre esse valor e o total de linhas modificadas pelo desenvolvedor naquele mesmo período, permitindo analisar quanto da produtividade total decorre da intervenção da IA.

Ao final desse processo, `get_code_lines_metrics` retorna uma série temporal

```

1 df_commit_metrics = pd.DataFrame(
2     [{"metrics": c, "date": c.date} for c in raw_commit_metrics]
3 )
4 df_copilot_code_metrics = pd.DataFrame(
5     [{"metrics": c, "date": c.date} for c in
6         raw_copilot_code_metrics],
7     columns=["metrics", "date"]
8 )
9 df_copilot_code_metrics["date"] = pd.to_datetime(
10     df_copilot_code_metrics["date"], errors="coerce"
11 )
12 grouped_commit_metrics = df_commit_metrics.groupby(
13     pd.Grouper(key="date", freq=period)
14 )
15
16 grouped_copilot_code_metrics = df_copilot_code_metrics.groupby(
17     pd.Grouper(key="date", freq=period)
18 )

```

Figura 20 – Trecho inicial de `get_code_lines_metrics`

consolidada que relaciona a produtividade medida em linhas de código ao uso da ferramenta de inteligência artificial ao longo do mesmo período.

Por sua vez, a função `get_commit_metrics` segue uma lógica semelhante ao método de cálculo baseado em linhas de código, porém adaptada ao escopo específico dessa métrica. Nesse modo, o sistema não considera o volume de linhas modificadas, mas sim o total de *commits* realizados em cada intervalo temporal. Quando o usuário opta por visualizar as métricas em termos de *commits*, o cálculo segue um fluxo mais simples, conforme ilustrado no trecho de código da Figura 21. Os dados brutos são agrupados segundo a periodicidade selecionada (diária, semanal, mensal ou outro período permitido), e para cada grupo o método contabiliza o número total de *commits*, identifica os autores presentes e registra as linguagens utilizadas naquele intervalo.

Embora o método também realize o agrupamento das métricas provenientes do *Copilot*, não há uma correlação direta entre sugestões de IA e número de *commits*, uma vez que a API do *GitHub Copilot* não expõe informações relacionadas ao ato de realizar um *commit*.

A saída consolidada desse método é composta por objetos da classe `CommitMetricsData`, cuja estrutura é apresentada na Figura 22.

Em ambos os modos de cálculo — seja baseado em linhas de código ou em número de *commits* — o resultado final é encapsulado em um objeto `CalculatedMetrics`. Esse objeto reúne o período de agregação, as linguagens envolvidas e a lista de estruturas de dados (`CodeLineMetricsData` ou `CommitMetricsData`) geradas no processo. Após a conclusão do cálculo, esses dados são serializados e enviados pela API ao frontend, onde

```
1 for period_final_date, commit_metrics_df in
    grouped_commit_metrics:
2     authors = []
3     for commit_metrics in commit_metrics_df["metrics"]:
4         if commit_metrics.language not in response.languages:
5             response.languages.append(commit_metrics.language)
6         if commit_metrics.author.name not in authors:
7             authors.append(commit_metrics.author.name)
8
9     response.data.append(
10         CommitMetricsData(
11             initial_date=period_final_date.to_period(period)
12                 .start_time.to_pydatetime(),
13             final_date=period_final_date.to_pydatetime(),
14             total_commits=len(commit_metrics_df["metrics"]),
15             percentage_copilot_suggestions_accepted=0,
16             number_of_authors=len(authors)
17         )
18     )
```

Figura 21 – Trecho de `get_commit_metrics`

```
1 class CommitMetricsData(BaseModel):
2     initial_date: datetime
3     final_date: datetime
4     total_commits: int
5     number_of_authors: int
```

Figura 22 – Estrutura da Classe `CommitMetricsData`

são utilizados para construir gráficos, indicadores e demais visualizações apresentadas ao usuário. Dessa forma, o backend atua como o núcleo analítico do sistema, responsável por consolidar e correlacionar informações de produtividade.

5.3.2.3 Language analytics

A seção de análise por linguagem apresenta a visualização do histórico de uso do *GitHub Copilot* segmentado por linguagem de programação. O usuário pode aplicar filtros por intervalo de datas e por linguagens a ser incluídas. A Figura 23, a seguir, ilustra o gráfico mencionado.

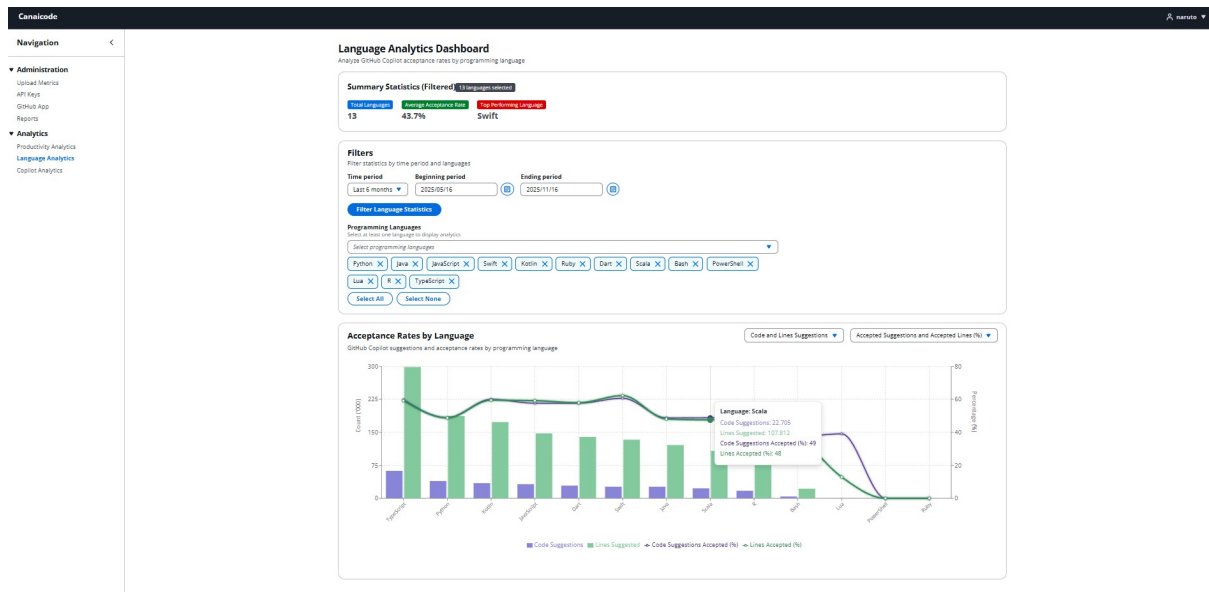


Figura 23 – Gráfico de *Language Analytics*

O gráfico dessa seção exibe quatro métricas: a quantidade de sugestões (em roxo), a quantidade de linhas de código (em verde), bem como a quantidade de sugestões e linhas de código que foram aceitas³.

O usuário pode, ainda, selecionar visualizar apenas as métricas relacionadas a sugestões ou apenas as referentes às linhas de código, resultando em uma representação mais simplificada, conforme ilustrado na Figura 24, abaixo.

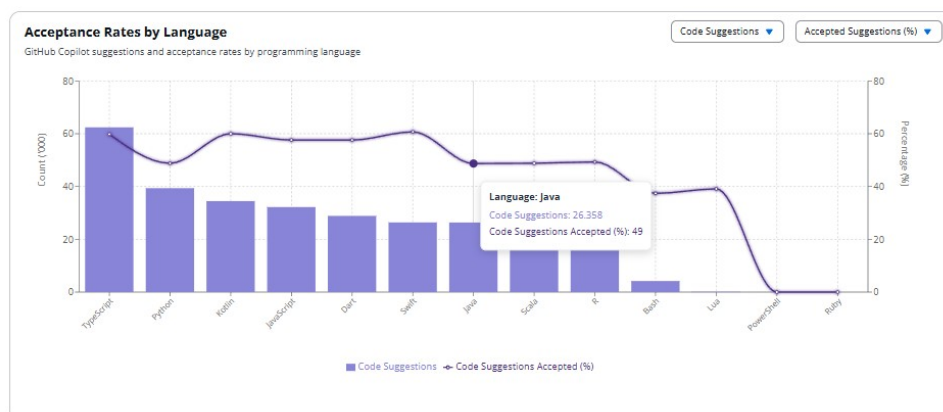


Figura 24 – Gráfico de *Language Analytics* simplificado

Na parte superior da tela, conforme já mostrado na Figura 23, é exibido um resumo contendo o número total de linguagens utilizadas no período analisado, a média geral de

³ As porcentagens auxiliam na identificação de linguagens cujas sugestões são mais frequentemente aceitas pelos desenvolvedores, sugerindo maior aderência ao uso da ferramenta. Contudo, o valor percentual isolado pode gerar interpretações equivocadas: uma única sugestão aceita, por exemplo, resultaria em 100% de aceitação, mas sem relevância estatística.

aceitação de sugestões e a linguagem com melhor desempenho no intervalo selecionado, respeitando os filtros de data aplicados. A definição de “melhor desempenho” considera a linguagem com maior fração de código produzido por IA.

No *backend*, assim como as métricas de *Productivity Analytics*, apresentadas anteriormente, essas métricas são geradas sob demanda, isto é, são calculadas em tempo de execução.

O sistema, ao obter as métricas do assistente de código *GitHub Copilot* para o período especificado, cria uma estrutura de dados em formato de dicionário, na qual cada chave representa uma linguagem de programação. O valor associado a cada chave é outro dicionário contendo quatro métricas: total de sugestões, total de sugestões aceitas, total de linhas sugeridas e total de linhas aceitas, todas inicialmente com valor zero. Em seguida, o sistema percorre a lista de métricas brutas, acumulando os valores correspondentes para cada linguagem. Após esse agrupamento, é realizada uma segunda iteração para o cálculo das porcentagens de aceitação de sugestões e de linhas. O resultado final consiste em uma lista consolidada, contendo, para cada linguagem, as quantidades absolutas e percentuais de sugestões e linhas aceitas, que são então retornadas ao painel de visualização.

5.3.2.4 Copilot Metrics

A tela de *Copilot Usage Analytics* disponibiliza ao gestor duas visualizações complementares para o acompanhamento do uso do *GitHub Copilot* na organização: (i) a análise da quantidade de usuários ativos e (ii) a análise da intensidade de utilização da ferramenta ao longo do tempo.

A primeira visualização apresenta o histórico diário da quantidade de usuários distintos que utilizaram o *GitHub Copilot*. Esse gráfico exibe duas métricas principais: o número de usuários do assistente de código — isto é, aqueles que aceitaram ao menos uma sugestão de código da ferramenta no respectivo dia (em verde) — e o número de usuários do *chat* integrado da ferramenta (em roxo). Essa visualização conta com filtro de data, permitindo restringir o período analisado conforme o interesse do usuário, de modo semelhante ao que ocorre na tela de produtividade geral.

A segunda visualização da tela corresponde à análise de intensidade de uso das sugestões do *Copilot*, que pode ser agregada em diferentes períodos — semanal, mensal, trimestral ou anual — de acordo com o nível de detalhamento desejado, conforme ilustrado na Figura 26. Essa seção é composta por dois gráficos complementares. No primeiro, são apresentadas as métricas de linhas de código: para cada período selecionado, exibe-se o total de linhas sugeridas pelo *Copilot* (em roxo) e o total de linhas aceitas (em verde). No segundo gráfico, são mostradas as métricas de sugestões, representando o total de sugestões feitas pelo assistente (em rosa) e o total de sugestões aceitas (em verde).

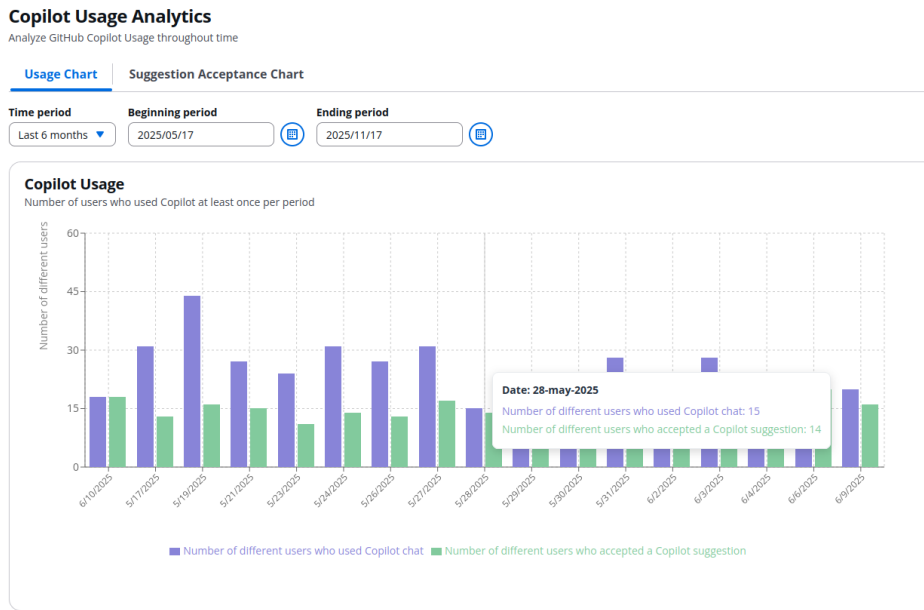


Figura 25 – Gráfico de quantidade de usuários diária.

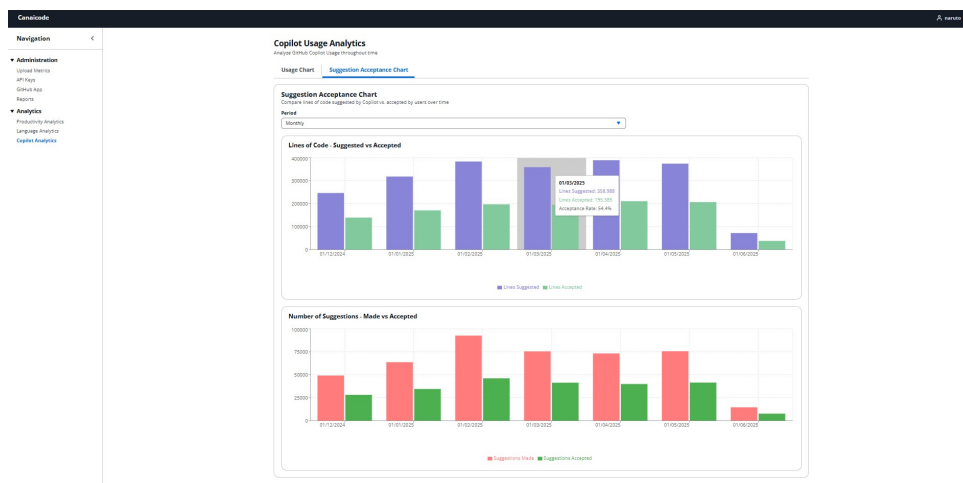


Figura 26 – Gráfico de aceitação de sugestões do *Copilot*.

No *backend*, as métricas exibidas na tela de *Copilot Usage Analytics* são calculadas sob demanda, de modo semelhante às demais seções do sistema. Para isso, são definidos dois métodos principais: o primeiro responsável pelo cálculo das métricas relacionadas à quantidade de usuários da ferramenta e o segundo voltado ao agrupamento da intensidade de uso por períodos específicos.

O primeiro método, apresentado no trecho da Figura 27, é responsável por gerar as métricas de uso do *GitHub Copilot* com base na quantidade de usuários ativos por dia. Inicialmente, o sistema obtém, por meio do repositório de métricas do assistente de código, todos os registros do período especificado. Caso não existam registros, uma lista vazia é retornada. Em seguida, é criado um dicionário denominado `grouped_code_metrics`, em

que cada chave representa uma data e o valor associado contém o número total de usuários que utilizaram o assistente de código naquele dia. O algoritmo percorre a lista de métricas, somando os valores de usuários correspondentes a cada data.

Após o agrupamento, é construída uma lista de objetos do tipo `CopilotUsersMetrics`, contendo para cada data a quantidade total de usuários do assistente de código.

Em um segundo momento, o método realiza procedimento análogo para os dados referentes ao *chat* do *Copilot*. São consultadas as métricas no repositório correspondente e agrupadas em um novo dicionário (`grouped_chat_metrics`), que mantém a estrutura de associação entre datas e número de usuários.

Por fim, as métricas do *chat* são integradas à lista de resposta, localizando o índice correspondente à data de cada registro e atribuindo o valor do total de usuários de *chat*. Dessa forma, o método retorna uma lista consolidada contendo, para cada data do intervalo analisado, a quantidade total de usuários do assistente de código e de usuários do *chat*, possibilitando a geração do gráfico de evolução diária de uso.

```

1 raw_copilot_code_metrics = self.copilot_code_metrics_repository.
  listByUserId(user_id, initial_date, final_date)
2
3 if (not raw_copilot_code_metrics):
4     return []
5
6 grouped_code_metrics: DefaultDict[datetime, Dict[str, int]] =
  DefaultDict(lambda: {
7     "total_users": 0,
8 })
9
10 for metric in raw_copilot_code_metrics:
11     date = metric.date
12     grouped_code_metrics[date]["total_users"] += metric.total_users
13
14 response: List[CopilotUsersMetrics] = []
15 for date_code_metrics, values_code_metrics in grouped_code_metrics.
  items():
16     response.append(CopilotUsersMetrics(
17         date=date_code_metrics,
18         total_code_assistant_users=values_code_metrics["total_users"]
19     ),
20     total_chat_users=0
21 )
22
23 raw_copilot_chat_metrics = self.copilot_chat_metrics_repository.
  listByUserId(user_id, initial_date, final_date)
24
25 grouped_chat_metrics: DefaultDict[datetime, Dict[str, int]] =
  DefaultDict(lambda: {
26     "total_users": 0,
27 })
28
29 for chat_metric in raw_copilot_chat_metrics:
30     date = chat_metric.date
31     grouped_chat_metrics[date]["total_users"] += chat_metric.
  total_users
32
33 for date_chat_metrics, values_chat_metrics in grouped_chat_metrics.
  items():
34     index = next(
35         i
36         for i, data in enumerate(response)
37         if data.date == date_chat_metrics
38     )
39     response[index].total_chat_users = values_chat_metrics["
  total_users"]
40
41 return response

```

Figura 27 – Trecho do método responsável pela geração de métricas de uso do *GitHub Copilot* por quantidade de usuários

O segundo método, apresentado no trecho da Figura 28, é responsável por calcular as métricas de intensidade de uso do *GitHub Copilot* de acordo com o período de agregação selecionado pelo usuário — que pode ser semanal, mensal, trimestral ou anual. Nesse caso,

as métricas brutas são inicialmente convertidas para um *DataFrame* da biblioteca **pandas**, contendo as colunas de data e o objeto de métricas associado. Em seguida, realiza-se o agrupamento das métricas com o método `groupby()`, utilizando a função `pd.Grouper()` com o parâmetro de frequência definido conforme o período desejado.

Para cada grupo resultante, o sistema calcula quatro métricas principais:

- o total de sugestões aceitas (`total_code_acceptances`);
- a porcentagem relativa de sugestões aceitas (`percentage_code_acceptances`);
- o total de linhas aceitas (`total_lines_accepted`);
- a porcentagem relativa de linhas aceitas (`percentage_lines_accepted`).

Esses cálculos são realizados por meio da classe auxiliar **MetricsCalculator**, que abstrai a lógica de soma e cálculo percentual, recebendo como entrada a lista de métricas pertencentes a cada grupo temporal.

Por fim, o método instancia objetos do tipo **CopilotMetricsByPeriod**, contendo a data inicial e final do período, bem como os valores absolutos e percentuais calculados. Essa lista de objetos é retornada ao *frontend*, onde é utilizada para a renderização dos gráficos de intensidade de uso.

```

1 df_copilot_code_metrics = pd.DataFrame(
2     [{"metrics": c, "date": c.date} for c in
3         raw_copilot_code_metrics],
4     columns=["metrics", "date"]
5 )
6 grouped_copilot_code_metrics = df_copilot_code_metrics.groupby(
7     pd.Grouper(key="date", freq=period)
8 )
9
10 response: List[CopilotMetricsByPeriod] = []
11
12 for (
13     copilot_period_final_date,
14     copilot_code_metrics_df,
15 ) in grouped_copilot_code_metrics:
16     total_code_acceptances = MetricsCalculator.
17         calculate_gross_use_of_AI(
18         copilot_code_metrics_df["metrics"].to_list()
19     )
20     percentage_code_acceptances = MetricsCalculator.
21         calculate_relative_use_of_AI(
22         copilot_code_metrics_df["metrics"].to_list()
23     )
24     total_lines_accepted = MetricsCalculator.
25         calculate_gross_use_of_AI_lines(
26         copilot_code_metrics_df["metrics"].to_list()
27     )
28     percentage_lines_accepted = MetricsCalculator.
29         calculate_relative_use_of_AI_lines(
30         copilot_code_metrics_df["metrics"].to_list()
31     )
32     response.append(CopilotMetricsByPeriod(
33         period_final_date=copilot_period_final_date.
34             to_pydatetime(),
35         period_initial_date=copilot_period_final_date.to_period(
36             period
37         ).start_time.to_pydatetime(),
38         total_code_acceptances=total_code_acceptances,
39         percentage_code_acceptances=percentage_code_acceptances,
40         total_lines_accepted=total_lines_accepted,
41         percentage_lines_accepted=percentage_lines_accepted
42     ))

```

Figura 28 – Trecho do método responsável pela geração de métricas de intensidade do uso do *GitHub Copilot* por período

5.3.3 Registro e Controle de Dados por Usuário

O sistema foi projetado para operar com base em um modelo de autenticação individualizada, de modo que cada usuário possua acesso exclusivo às suas informações e

métricas. Essa abordagem garante a privacidade dos dados.

Nesta seção, são descritos os mecanismos de registro e autenticação de usuários, as políticas de controle de acesso aplicadas às rotas protegidas e a estrutura de separação lógica dos dados no banco de dados, assegurando que cada usuário visualize apenas as informações sob sua responsabilidade.

5.3.3.1 Login e Registro

O processo de registro e autenticação é responsável por gerenciar a criação e o controle das credenciais de acesso ao sistema. O módulo de autenticação foi desenvolvido com base em uma arquitetura de rotas seguras, integrando mecanismos de verificação de identidade e geração de tokens de sessão.

Durante o registro, o usuário fornece informações básicas — como nome, e-mail e senha, além de dados referentes à sua equipe de gestão — que são armazenadas no banco de dados, com o uso de técnicas de *hashing* para proteção das credenciais. Após o registro, o processo de *login* realiza a validação dos dados informados e emite um token de autenticação, necessário para acessar as demais funcionalidades do sistema.

As Figuras 29 e 30 ilustram as interfaces acessadas pelo usuário para efetuar o registro e *login*.

Canaicode Dashboard
Home
Login
Register

User Register

Fill in all the required information to create your account.

Full Name

Your complete name (first and last name required)

Enterprise Name

Company or business name (optional)

Email

Your email address

Cellphone

Brazilian cellphone number (11 digits)

CPF/CNPJ

Your CPF (11 digits) or company CNPJ (14 digits)

Username

3-50 characters, lowercase letters and numbers only

Password

At least 8 characters with uppercase, lowercase, and number

Register

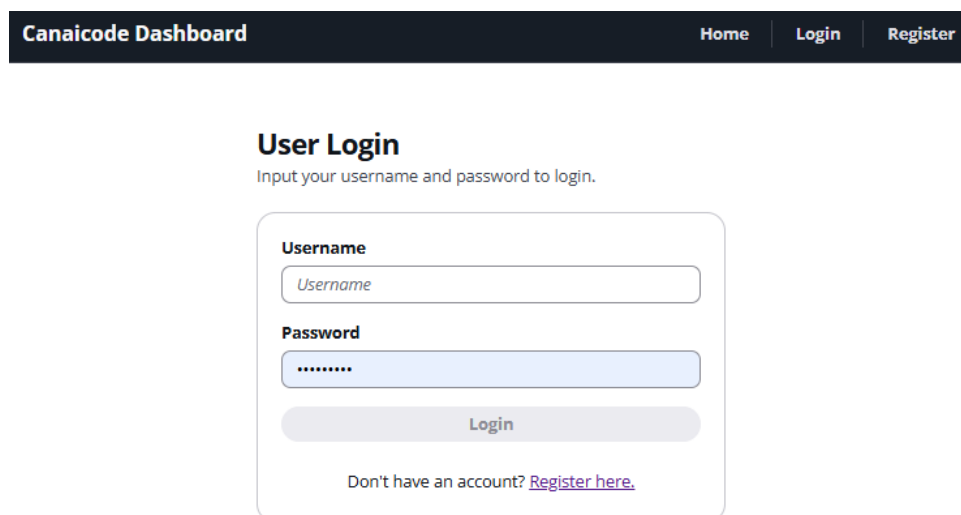
Already have an account? [Login here.](#)

Figura 29 – Tela de registro de usuário.

De tal maneira, a tabela *users* do banco de dados é populada a partir do seguinte esquema, na tabela 6.

Tabela 6 – Estrutura de usuários

Coluna	Tipo de Dado	Legenda
id	text	Identificador único
username	text	Nome de usuário
hashed_password	text	Senha criptografada
created_at	datetime	Data de criação
cpf_cnpj	text	Documento do usuário ou empresa
cellphone	text	Celular do usuário
email	text	Email do usuário
full_name	text	Nome completo do usuário
enterprise_name	text	Nome da empresa associada ao usuário



The image shows a web interface for a 'Canaicode Dashboard'. At the top, there is a dark navigation bar with the text 'Canaicode Dashboard' on the left and three links: 'Home', 'Login', and 'Register' on the right. Below the navigation bar, the main content area is titled 'User Login' in a bold font. Underneath the title, a subtitle reads 'Input your username and password to login.' The login form is enclosed in a light gray rounded rectangle. It contains two input fields: 'Username' with a placeholder text 'Username' and 'Password' with a placeholder of seven dots. Below these fields is a 'Login' button. At the bottom of the form, there is a link that says 'Don't have an account? [Register here.](#)'

Figura 30 – Tela de *login* de usuário.

Por sua vez, o processo de autenticação é responsável por validar as credenciais fornecidas pelo usuário e emitir um *token* de acesso (*access token*) que será utilizado para autorizar o uso das rotas protegidas do sistema.

Durante a autenticação, o sistema busca o usuário no repositório de dados a partir do nome informado e, em seguida, valida a senha provida pelo usuário. Caso a combinação de credenciais seja inválida, é lançada uma exceção do tipo `HTTPException`, interrompendo o processo de autenticação.

Uma vez confirmada a identidade do usuário, é gerado um *token* de autenticação do tipo JWT (*JSON Web Token*), criado pelo método `create_access_token`. Esse *token* encapsula informações essenciais do usuário — como o identificador único (`user_id`) e o nome de usuário (`sub`) —, além de um campo de expiração (`exp`) que define a validade temporal do *token*.

O *token* gerado é assinado digitalmente utilizando a chave secreta `SECRET_KEY` e o algoritmo definido em `ALGORITHM`, de modo a assegurar a integridade e a autenticidade do conteúdo. Por fim, o *token* é retornado ao cliente em formato `Bearer`, devendo ser incluído no cabeçalho de autenticação de todas as requisições subsequentes a rotas protegidas.

5.3.3.2 Proteção de Rotas

Uma vez autenticado, o usuário passa a possuir um *token* de acesso válido, armazenado na sessão do navegador. Esse *token*, emitido no momento do *login*, deve ser incluído no *payload* de todas as requisições subsequentes a rotas protegidas do sistema.

A autenticação baseada em *token* permite que o servidor valide cada requisição individualmente, garantindo que apenas usuários devidamente autenticados e autorizados possam executar ações sensíveis, como o envio ou a consulta de métricas.

A proteção das rotas ocorre em duas etapas complementares. A primeira corresponde à autenticação: o servidor verifica a validade e a integridade do *token* recebido, assegurando que ele foi emitido pelo serviço e que permanece dentro do período de validade. A segunda etapa corresponde à autorização, em que o sistema confirma se o usuário autenticado possui permissão para acessar ou manipular os dados solicitados.

Para isso, o conteúdo do *token* é decodificado e utilizado para identificar o usuário associado à requisição. Em seguida, o servidor compara essa identificação com o usuário-alvo da operação. Caso haja inconsistência (*token* inválido, expirado ou associado a um usuário diferente) a requisição é bloqueada e uma resposta de erro apropriada é retornada.

5.3.3.3 Separação de Dados por Usuário

Além do controle de autenticação e autorização, o sistema foi desenvolvido de modo a garantir a segregação lógica dos dados entre os diferentes usuários. Cada registro inserido no banco de dados é associado de forma direta ao identificador único (`user_id`) do usuário autenticado responsável pela operação.

Dessa forma, todas as consultas, cálculos e exibições de métricas realizadas pelo sistema são automaticamente filtradas com base nesse identificador. Essa arquitetura assegura que cada usuário visualize exclusivamente os dados por ele gerados, preservando a confidencialidade e a integridade da informação. A Figura 31 mostra um exemplo de uma rota protegida.

Essa camada adicional de controle reforça o modelo de segurança e privacidade adotado no sistema, sendo essencial para a correta operação em ambientes multiusuário.

5.3.4 Relatórios periódicos

A plataforma permite que os usuários configurem o envio automático de relatórios analíticos por e-mail, contendo gráficos e *insights* sobre o uso do *GitHub Copilot* e a produtividade dos desenvolvedores ao longo do tempo. Esse recurso visa oferecer uma visão consolidada e recorrente do desempenho da equipe, sem a necessidade de acessar manualmente o *dashboard*.

```
1 def listByUserId(  
2     self,  
3     user_id: str,  
4     initial_date: Optional[datetime] = None,  
5     final_date: Optional[datetime] = None,  
6     languages: Optional[List[str]] = None,  
7 ) -> List[CommitMetrics]:  
8     query = self.db.query(RawCommitMetrics)  
9  
10    if initial_date:  
11        query = query.filter(RawCommitMetrics.date >= initial_date)  
12  
13    if final_date:  
14        query = query.filter(RawCommitMetrics.date <= final_date)  
15  
16    if languages:  
17        query = query.filter(RawCommitMetrics.language.in_(languages  
18                               ))  
19  
20    records = query.filter(RawCommitMetrics.user_id == user_id).all  
21    ()  
22  
23    commit_metrics = map(  
24        lambda record: DatabaseRawCommitMetricsMapper.to_domain(  
25            record), records  
26    )  
27  
28    return list(commit_metrics)
```

Figura 31 – Trecho de código que exemplifica a filtragem de dados por identificador de usuário

5.3.4.1 Configuração de envio de relatórios

No frontend, o usuário pode cadastrar uma configuração personalizada de envio periódico dos relatórios, informando os e-mails que devem receber a mensagem, assim como o intervalo desejado: diário, semanal, mensal ou trimestral. Caso já exista uma configuração, o usuário pode atualizá-la, alterando os endereços de e-mail, o período ou mesmo excluir o envio automático.

No backend, essa funcionalidade é implementada como um *CRUD* simples, com uma entidade de **ReportConfig**, que persiste essas informações e é utilizada posteriormente pelo *worker* responsável pelo envio. A *API* expõe os *endpoints* necessários para permitir ao usuário realizar todas essas operações.

No banco de dados, a entidade **ReportConfig** é representada da seguinte maneira:

5.3.4.2 *Worker* de envio de relatórios

O envio automático dos relatórios é orquestrado por um *worker*, executado periodicamente utilizando o **APScheduler**. A função principal é responsável por iterar sobre

Tabela 7 – Configuração do envio de relatórios

Coluna	Tipo de Dado	Legenda
id	text	Identificador único
emails	text	E-mails cadastrados, separados por vírgula
period	text	Período para envio
user_id	text	Identificador do usuário associado
created_at	datetime	Data de inserção da configuração

todas as configurações cadastradas e, com base na periodicidade definida e na data atual, acionar os métodos que constroem os gráficos e coletam os dados dos usuários inativos.

Cada um desses métodos de geração utiliza os cálculos de agregação de métricas já existentes no backend, explicados nas subseções anteriores, para obter métricas processadas de produtividade, uso do *Copilot* ou quantidade de usuários.

As métricas para cada tipo de relatório estão detalhadas a seguir:

- Diário e Semanal:
 - Produtividade nas últimas 10 semanas (linhas de código).
 - Produtividade nos últimos 6 meses (linhas de código).
 - Uso do Copilot por linguagem (última semana).
 - Uso do Copilot por dia (última semana).
 - Quantidade de usuários ativos no Copilot por dia (última semana).
 - Lista dos usuários inativos há pelo menos 30 dias.
- Mensal:
 - Produtividade nas últimas 10 semanas (linhas de código).
 - Produtividade nos últimos 6 meses (linhas de código).
 - Uso do Copilot por linguagem (último mês).
 - Uso do Copilot por semana (último mês).
 - Quantidade de usuários ativos no Copilot por dia (último mês).
 - Lista dos usuários inativos há pelo menos 30 dias.
- Trimestral:
 - Produtividade nas últimas 10 semanas (linhas de código).
 - Produtividade nos últimos 6 meses (linhas de código).
 - Uso do Copilot por linguagem (último trimestre).

- Uso do Copilot por semana (último trimestre).
- Quantidade de usuários ativos no Copilot por dia (último mês).
- Lista dos usuários inativos há pelo menos 90 dias.

A geração dos gráficos é realizada pelo método `make_graph`. Esse método recebe como entrada uma lista de métricas, que são convertidas em objetos `DataFrame` utilizando a biblioteca `pandas`. Cada `DataFrame` é então processado com o `matplotlib` para criar visualizações que facilitam a interpretação do desempenho da equipe ou dos projetos ao longo do tempo. Então, os gráficos são salvos como *.PNGs*.

A função foi projetada para ser flexível e adaptativa, distinguindo automaticamente entre dados temporais (por exemplo, datas) e categóricos (por exemplo, nomes de repositórios). Quando o eixo *X* contém valores temporais, é aplicada formatação para exibir apenas datas relevantes e evitar sobreposição de rótulos. Já para dados categóricos, os rótulos são ajustados em diagonal, mantendo a legibilidade mesmo em conjuntos extensos. Além disso, todos os gráficos são renderizados em *buffer* de memória (via `io.BytesIO`), permitindo que sejam embutidos no e-mail sem a necessidade de salvar arquivos temporários.

Para organizações que possuem um `GitHub App` configurado, o *worker* também é executada uma chamada à *API* de *billing* do *GitHub Copilot*, por meio do método `get_inactive_users`, ilustrado no trecho 32.

Esse método autentica o acesso utilizando um *JWT* temporário, obtém a lista de *seats* da organização e combina a data da última atividade com o limite configurado no período do relatório, retornando os *logins* dos usuários que não utilizaram o *Copilot* há pelo menos um dado período de tempo, definido pelo usuário. Esse dado é incorporado ao corpo do e-mail como uma lista simples.

Após gerar os gráficos e identificar os usuários inativos, o método `send_email` faz a montagem e o envio da mensagem ao destinatário. O conteúdo do e-mail é estruturado em *HTML*. Os gráficos são embutidos como imagens *inline* utilizando identificadores de conteúdo, o que permite que sejam exibidos diretamente no corpo da mensagem, sem depender de anexos.

Além dos gráficos, o e-mail inclui uma lista dos usuários inativos (se configurado desta forma), exibida como texto simples logo abaixo das visualizações. O envio é realizado por meio do protocolo *SMTP* (`smtpplib.SMTP_SSL`), utilizando as credenciais configuradas no ambiente do sistema.

As Figura 33 e 34, a seguir, apresentam um exemplo de relatório enviado pelo *worker*, gerado automaticamente pelo sistema, contendo os gráficos. Essa visualização demonstra a integração das métricas analíticas com a comunicação por e-mail, oferecendo uma forma prática e acessível de monitorar o uso do *GitHub Copilot* dentro da organização.

```

1 def get_inactive_users(self, github_app: GitHubApp, days_inactive:
2   int) -> List[str]:
3   try:
4       now = int(time.time())
5
6       private_key = self.fernet.decrypt(github_app.
7         private_key_encrypted.encode()).decode()
8
9       payload: Dict[str, str | int] = {
10         "iat": now,
11         "exp": now + (10 * 60), # JWT valid for 10 minutes
12         "iss": github_app.app_id,
13       }
14       jwt_token = jwt.encode(payload, private_key, algorithm="RS256")
15
16       headers = {
17         "Authorization": f"Bearer {jwt_token}",
18         "Accept": "application/vnd.github+json",
19       }
20       token_url = f"https://api.github.com/app/installations/{
21         github_app.installation_id}/access_tokens"
22       token_response = requests.post(token_url, headers=headers)
23       token_response.raise_for_status()
24       access_token = token_response.json()["token"]
25
26       headers = {
27         "Authorization": f"Bearer {access_token}",
28         "Accept": "application/vnd.github+json",
29       }
30       seats_url = f"https://api.github.com/orgs/{github_app.
31         organization_name}/copilot/billing/seats"
32       seats_response = requests.get(seats_url, headers=headers)
33       seats_response.raise_for_status()
34       data = seats_response.json()
35
36       inactive_users: List[str] = []
37       limit = datetime.now(timezone.utc) - timedelta(days=
38         days_inactive)
39
40       for seat in data.get("seats", []):
41         last_active = seat.get("last_activity_at")
42         if last_active:
43             last_active_dt = datetime.fromisoformat(last_active.
44               replace("Z", "+00:00"))
45             if last_active_dt < limit:
46                 inactive_users.append(seat["user"]["login"])
47
48       return inactive_users
49   except Exception as e:
50       logger.error(f"Error fetching inactive users: {e}")
51       return []

```

Figura 32 – Worker para envio de relatório de métricas periódico - usuários inativos

Figura 33 – E-mail de *reports* (I/II).

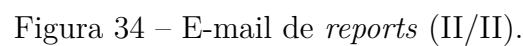
5.3.5 Hospedagem e execução

A hospedagem do sistema está sendo feita em um ambiente cloud AWS. Visando facilitar a manutenção, escolheu-se hospedar todo o sistema, tanto frontend quanto backend, em uma única máquina virtual a partir do serviço EC2.

Usa-se o utilitário de sistema *nginx* para efetuar a *proxy* reversa HTTP, a qual fica responsável por direcionar tráfego de diferentes caminhos para diferentes portas de rede.

No caso, tráfego HTTP para a rota "/" é direcionado para o frontend, enquanto que para a rota "/api" é direcionado para o backend.

Para a atualização do sistema, utiliza-se scripts *.sh* que atualizam o código para sua versão mais recente e reiniciam os processos de backend e frontend que estão em execução.



As visualizações produzidas pelo sistema permitem que um gestor observe padrões de comportamento relacionados tanto à produtividade dos desenvolvedores quanto ao uso do *GitHub Copilot*. Ao combinar métricas de *commits*, linhas de código e indicadores de adoção de IA generativa, o sistema fornece uma perspectiva que conecta atividade de desenvolvimento e uso de tecnologias de suporte.

Paralelamente, a adoção de soluções que dependem de informações internas pode representar um desafio para muitas empresas, especialmente quando envolve dados sensíveis

ou acesso direto a repositórios corporativos. Por isso, a questão da confidencialidade tornou-se um aspecto central no desenvolvimento do sistema. As soluções propostas buscam garantir que o uso da ferramenta não exija o compartilhamento de informações privadas ou estratégicas, preservando a segurança e a autonomia do usuário.

5.4.1 *Productivity Analytics*

A seção *Productivity Analytics* apresenta gráficos que relacionam métricas tradicionais de produtividade — como número de *commits* ou linhas de código modificadas — com o uso do *GitHub Copilot*. Embora a *API* do Copilot não ofereça uma métrica diretamente correlacionada ao número de *commits*, essa visualização fornece uma perspectiva complementar que permite ao gestor:

- comparar períodos de maior ou menor volume de entregas com períodos de maior ou menor uso do *Copilot*;
- observar cadências de atividade ao longo do tempo, identificando fases de implementação intensiva, estabilização ou planejamento;
- relacionar oscilações de produtividade com dados de intensidade de uso apresentados na subseção 5.4.3, a seguir.

O modo de cálculo adotado permite observar a dinâmica temporal dos *commits*, funcionando como um indicador indireto da atividade de desenvolvimento. Mesmo não havendo uma associação direta entre número de *commits* e sugestões aceitas do *Copilot*, a análise conjunta dos dois conjuntos de dados possibilita identificar tendências e potenciais correlações. Assim, *Productivity Analytics* torna-se uma ferramenta útil para avaliar, de forma exploratória, se o uso da IA acompanha aumentos ou reduções no ritmo de produção de código.

Além disso, o gráfico de *Productivity Analytics* de linhas de código é onde se pode ver mais diretamente a correlação entre a produtividade e uso da ferramenta de IA, uma vez que as métricas são casadas diretamente, linha a linha.

5.4.2 *Language Analytics*

A seção *Language Analytics* resume-se a comparação do uso do *GitHub Copilot* entre diferentes linguagens de programação. Essa análise é particularmente relevante em equipes que operam em múltiplas tecnologias.

O gráfico de *acceptance rates* por linguagem, ilustrado na Figura 26, permite ao gestor:

- identificar linguagens nas quais o *Copilot* apresenta maior taxa de aceitação de sugestões, sugerindo maior aderência da ferramenta àquela tecnologia;
- verificar linguagens com adoção reduzida, o que pode indicar menor familiaridade das equipes, baixa utilidade percebida ou limitações atuais do modelo para aquele ecossistema;
- aplicar filtros por intervalo de datas para analisar períodos específicos, como um *sprint*, *milestone*, projeto ou ciclo de desenvolvimento.

Essa perspectiva possibilita compreender como o *Copilot* se comporta em diferentes bases de código e pode orientar decisões estratégicas, como priorização de treinamentos, padronização tecnológica ou alocação de esforços em linguagens onde o ganho potencial pode ser maior.

5.4.3 *Copilot Usage Analytics*

A seção *Copilot Usage Analytics* apresenta duas visualizações complementares: o gráfico de usuários ativos por dia, que pode ser filtrada por intervalo de tempo, por exemplo, permitindo análises focada em um projeto específico, e o gráfico de intensidade de uso baseado em sugestões aceitas.

5.4.3.1 *Usage Chart*

O gráfico de usuários ativos auxilia o gestor a compreender padrões de adoção do *Copilot* por parte da equipe. A partir dessa visualização, é possível:

- identificar picos de adoção decorrentes da introdução da ferramenta em uma nova equipe ou projeto;
- analisar a velocidade com que novos usuários passam a utilizar o *Copilot*;
- observar tendências de engajamento, como crescimento progressivo, estabilização ou queda de uso.

5.4.3.2 *Suggestion Acceptance Chart*

O gráfico de *Suggestion Acceptance* fornece uma medida da intensidade efetiva de utilização da IA. Essa métrica permite ao gestor:

- identificar períodos de maior ou menor uso do *Copilot*;
- correlacionar picos de utilização com etapas de desenvolvimento mais intensas, como criação de novas funcionalidades;

- reconhecer fases de baixa aceitação, que podem indicar períodos de manutenção, refatoração ou menor dependência da ferramenta.

5.4.3.3 Análise integrada dos gráficos de *Copilot Analytics*

O cruzamento entre os dois gráficos — número de usuários e intensidade de uso — possibilita uma interpretação mais rica do comportamento da equipe. Por exemplo:

- aumentos no número de usuários podem não resultar em aumento proporcional na intensidade de uso, sugerindo que novos usuários ainda estão em fase de adaptação;
- usuários experientes podem manter níveis elevados de aceitação de sugestões independentemente do número total de usuários;
- discrepâncias entre crescimento de usuários e redução da intensidade podem indicar mudanças no tipo de trabalho desenvolvido naquele período.

Essa combinação de perspectivas fornece um panorama do ciclo de adoção da IA no contexto de um dado projeto de desenvolvimento de software.

5.4.4 Análises cruzadas

Ademais, pode-se tirar *insights* a partir da análise cruzada entre os gráficos disponibilizados. O cruzamento de dados de *Productivity Analytics*, que buscam indiciar se de fato há maior produtividade com o uso de ferramentas de IA, com os indicadores de *Copilot Usage Analytics* permite um diagnóstico sobre a maturidade de adoção dentro da equipe: é possível identificar se uma correlação fraca estaria associada a entrada de novos usuários, ao início de um projeto novo ou adoção de uma nova linguagem específica.

5.4.5 Confidencialidade

Diante da preocupação em evitar o fornecimento de dados sensíveis, no caso da coleta de métricas de commits, foram oferecidas duas alternativas compatíveis com esse princípio. Na versão manual, o processo ocorre inteiramente na máquina do usuário, por meio de um script local que percorre os repositórios e gera um arquivo Excel. Esse arquivo pode ser revisado e validado pelo próprio usuário antes de ser enviado à plataforma, garantindo total controle sobre os dados compartilhados.

Já o modo automático utiliza um GitHub Actions configurado pelo próprio usuário em seu repositório. A coleta e o processamento das métricas acontecem dentro do ambiente do próprio GitHub do usuário, e apenas as informações estritamente necessárias são enviadas ao sistema. Apesar do envio automatizado, a configuração é feita manualmente, permitindo que o usuário verifique previamente o conteúdo que será compartilhado.

Por fim, para a integração com o GitHub Copilot, o tutorial disponibilizado orienta o usuário a criar um perfil com permissão limitada, em modo somente leitura, restringindo-se exclusivamente aos dados de usabilidade do Copilot. Dessa forma, não há acesso a repositórios ou a qualquer outro tipo de informação sensível.

5.5 Documentação da *API*

Abaixo é exposta a lista completa das rotas disponíveis na *API* desenvolvida para o backend do sistema.

Method	Endpoint	Description	Auth
Authentication Routes			
POST	<code></register></code>	User registration with username, password, email, and profile information	No
POST	<code></login></code>	User login with OAuth2 password flow (username/password)	No
Commit Metrics Routes			
GET	<code></commit_metrics/upload_deprecated/{team_name}></code>	(Deprecated) Get commit metrics by team name and date	No
POST	<code></commit_metrics/upload></code>	Upload commit metrics from XLSX file for a specific user	Dual Auth
Copilot Metrics Routes			
POST	<code></copilot_metrics/upload></code>	Upload copilot metrics from JSON file for a specific user	Dual Auth
GET	<code></copilot_metrics/language/{user_id}></code>	Get copilot metrics grouped by programming language for a user	Token
GET	<code></copilot_metrics/period/{user_id}></code>	Get copilot metrics grouped by time period for a user	Token
GET	<code></copilot_metrics/users/{user_id}></code>	Get copilot metrics for multiple users in an organization	Token
Calculated Metrics Routes			
GET	<code></calculated_metrics/{user_id}></code>	Get calculated productivity metrics with custom period and filters	Token
GitHub App Routes			
POST	<code></github_app></code>	Register/create a GitHub app integration with organization details	Token
GET	<code></github_app/{user_id}></code>	Find the GitHub app for a specific user if exists	Token
DELETE	<code></github_app/{github_app_id}></code>	Delete a specific GitHub app	Token
API Key Management Routes			
POST	<code></api_keys></code>	Create a new API key for a user with optional expiration	Token
GET	<code></api_keys/{user_id}></code>	List all API keys for a specific user (keys are masked)	Token
DELETE	<code></api_keys/{key_id}></code>	Revoke/delete a specific API key	Token
Report Configuration Routes			
POST	<code></report_config></code>	Create a new report configuration with e-mails addresses and period	Token
GET	<code></report_config/{user_id}></code>	Find the report configuration for a specific user if exists	Token
PUT	<code></report_config/{report_config_id}></code>	Update a specific report configuration	Token
DELETE	<code></report_config/{report_config_id}></code>	Delete a specific report configuration	Token

Tabela 8 – Rotas da API

6 Validação

Este capítulo apresenta o processo de validação do sistema desenvolvido, realizado por meio de um questionário. O objetivo é verificar se as funcionalidades implementadas atendem aos requisitos propostos e se a experiência de uso está de acordo com o esperado. Para isso, descrevem-se o questionário aplicado, os critérios avaliados e as respostas fornecidas pelos validadores, permitindo analisar a conformidade do sistema com os objetivos do projeto.

6.1 Questionário de Validação do Sistema

Para validar o sistema desenvolvido e verificar seu alinhamento com os objetivos do trabalho, foi elaborado um questionário¹ estruturado em cinco blocos temáticos. O instrumento foi construído de forma a avaliar tanto aspectos técnicos — como funcionamento das funcionalidades essenciais — quanto aspectos de usabilidade e utilidade prática das informações apresentadas no *dashboard* e demais funcionalidades do sistema. Dessa forma, o questionário opera como uma ponte entre os requisitos definidos no projeto e a percepção de uso de um usuário real.

1. A seção sobre o **perfil do usuário** coleta informações contextuais sobre o respondente, como papel na equipe de desenvolvimento, tamanho da equipe e grau de adoção de ferramentas de IA. Essas perguntas permitem situar as respostas dentro da realidade operacional do usuário, garantindo que a análise considere diferentes níveis de experiência com *Git*, métricas de produtividade e uso do *GitHub Copilot*. Tal contextualização é importante, pois os objetivos do projeto envolvem avaliar o impacto de ferramentas de IA em equipes reais de desenvolvimento. Logo, compreender o perfil do validador ajuda a interpretar adequadamente seus julgamentos sobre o sistema.
2. A avaliação de **usabilidade e funcionalidade** aborda diretamente os objetivos centrais do projeto. A primeira parte deste bloco verifica se os processos de coleta de dados, tanto manual quanto automática, funcionam corretamente — aspectos que correspondem às metas de “coletar dados brutos dos repositórios *Git*” e “coletar dados de uso do *GitHub Copilot* via *API* oficial”. Já a segunda parte avalia a clareza, relevância e potencial analítico das visualizações. Essa avaliação está diretamente associada ao objetivo de “apresentar as informações em um *dashboard* dinâmico que permita interpretar métricas e compreender seu impacto na produtividade”. Cada

¹ Disponível em: <https://forms.gle/HMNxNLwvUVD8Qd7j9>.

métrica — linhas de código, *commits*, aceitação de sugestões, número de usuários e análise de linguagens — é examinada quanto à sua utilidade, permitindo verificar se a ferramenta cumpre seu papel informativo.

3. A seção sobre os **reports periódicos** avalia uma funcionalidade complementar, mas alinhada aos objetivos do sistema: o envio automático de relatórios. Esse bloco verifica se o processo de assinatura e recebimento dos relatórios é simples e se os gráficos enviados são claros e úteis para gestores, reforçando o compromisso do projeto em facilitar o acompanhamento contínuo do impacto do *Copilot* na produtividade da equipe.
4. Na **avaliação qualitativa**, o questionário abre espaço para que o validador descreva dificuldades, sugestões e percepções sobre a utilidade das informações fornecidas. Esse bloco é essencial para identificar possíveis lacunas entre os objetivos da solução e a experiência prática de uso, fornecendo *feedback* direto sobre a contribuição do *dashboard* para a análise da relação entre produtividade e uso do *GitHub Copilot*.
5. Por fim, a **avaliação geral** sintetiza a percepção global do usuário quanto à satisfação e intenção de uso futuro. Esses elementos funcionam como indicadores finais de aderência aos objetivos do projeto, especialmente no que se refere à criação de uma ferramenta útil, acessível e relevante para o contexto de desenvolvimento de software.

Assim, o questionário desenvolvido guia a validação das funcionalidades implementadas, verificando se o sistema cumpre seus objetivos fundamentais: coletar, tratar e apresentar métricas que auxiliem na análise do impacto de ferramentas de IA no processo de desenvolvimento.

6.2 Resultados

Devido a restrições internas de segurança e confidencialidade, a validação do sistema enfrentou um desafio: algumas empresas não puderam autorizar o compartilhamento dos dados reais de seus repositórios e do uso do *GitHub Copilot*. Como consequência, apenas um dos validadores pôde utilizar o sistema com dados reais, enquanto os demais participantes realizaram a avaliação utilizando dados preparados para simular cenários típicos de uso.

Dessa forma, esta seção apresenta os resultados organizados em duas partes: primeiro, a análise detalhada das respostas fornecidas pelo usuário que operou o sistema com dados reais e, em seguida, um resumo consolidado das respostas de todos os validadores.

6.2.1 Utilização Com Dados Reais

O validador que utilizou o sistema com dados reais atua como CTO da empresa Opus Software². Ele gerencia uma equipe composta por 4 a 6 desenvolvedores, dos quais entre 75% e 100% utilizam o *GitHub Copilot*. Segundo o próprio validador, a equipe utiliza a ferramenta há mais de um ano, embora alguns novos membros contratados recentemente estejam em fase inicial de treinamento e tenham sido introduzidos ao *Copilot* há menos de três meses.

No que diz respeito às funcionalidades de coleta de dados, o usuário atribuiu nota máxima ao processo de *upload* manual, destacando que a operação ocorreu sem dificuldades. As funcionalidades de *upload* automático, entretanto, não foram utilizadas. Em relação ao funcionamento geral do sistema, o validador atribuiu nota máxima ao tempo de resposta e uma avaliação intermediária para as mensagens de erro e *feedback*, sugerindo espaço para aprimoramentos na comunicação de falhas e orientações ao usuário.

Em respeito à clareza e utilidade das visualizações do *dashboard*, observou-se uma avaliação majoritariamente positiva. Sobretudo, o validador deu nota máxima para a categoria *nota geral* em todos os gráficos, com exceção à Correlação de *Pearson*, que recebeu nota 4. Na categoria *facilidade de interpretação*, tanto a Correlação de *Pearson* quanto o gráfico de análise de linguagens de programação receberam nota 4, enquanto os demais foram avaliados com nota 5. Em relação à relevância das informações para análise de produtividade, a Correlação de *Pearson* e o gráfico de uso do *Copilot* por número de usuários receberam nota 3, e os demais, nota 4. Por fim, quanto ao potencial de geração de *insights*, a Correlação de *Pearson* obteve nota 2, enquanto todos os gráficos receberam nota 4. Em suma, esses resultados sugerem que, embora a maioria das visualizações seja considerada clara e útil, métricas de correlação exigem maior contextualização ou refinamento para facilitar sua interpretação.

No bloco referente aos *reports* periódicos, o usuário atribuiu nota máxima a todos os itens avaliados. O processo de configuração foi considerado simples e intuitivo, e os gráficos enviados automaticamente foram percebidos como claros, legíveis e úteis para acompanhamento continuado.

Na avaliação qualitativa, o validador relatou ter encontrado duplicação de dados do *Copilot* em datas repetidas nos arquivos JSON exportados da *API*, mas ressaltou que, após a identificação do problema, as informações apresentaram coerência. Quanto ao uso das métricas, destacou que diversos fatores influenciam a produtividade e que, portanto, não é simples estabelecer correlações diretas. Ainda assim, reconheceu que o sistema é valioso por evidenciar o nível real de adoção da ferramenta pela equipe. Por fim, sugeriu como melhoria a possibilidade de compartilhar o *dashboard* com outros colegas da empresa,

² Site da empresa: <https://www.opus-software.com.br/>.

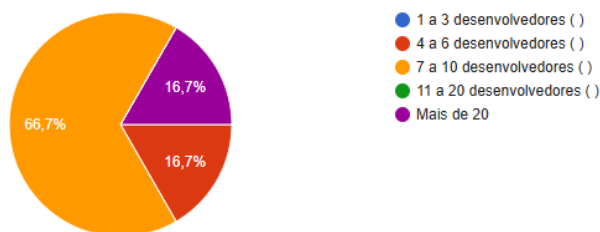


Figura 35 – Tamanho das equipes de desenvolvimento

ampliando seu potencial de uso.

Na avaliação geral, o usuário atribuiu nota máxima tanto para a satisfação geral com o sistema quanto para a probabilidade de utilizá-lo novamente, indicando uma experiência amplamente positiva.

Em síntese, a validação com dados reais demonstra que o sistema atende de maneira eficaz aos seus objetivos principais, especialmente no tocante à clareza das visualizações, à utilidade prática das métricas e ao envio automático de *reports*. As observações registradas oferecem direcionamentos pontuais para aprimoramentos, sobretudo na interpretação de métricas avançadas e na comunicação de erros, sem comprometer a impressão geral de utilidade do sistema.

6.2.2 Avaliação Geral dos Usuários

A fase de testes envolveu seis participantes, dos quais, apenas um pôde operar a ferramenta com informações provenientes de um ambiente real, enquanto os demais conduziram a avaliação utilizando conjuntos de dados sintéticos.

6.2.2.1 Perfil dos Respondentes

Entre os seis participantes, quatro atuam como desenvolvedores plenos, um como *Tech Lead* e um como CTO. Em relação ao tamanho das equipes de desenvolvimento, a distribuição dos respondentes foi concentrada em equipes de 7 a 10 pessoas, conforme é indicado no gráfico 35.

A adoção do GitHub Copilot entre essas equipes é elevada: todos os respondentes indicaram que entre 75% e 100% dos desenvolvedores utilizam a ferramenta. Quanto ao tempo de uso, duas equipes utilizam o Copilot entre 6 meses e 1 ano, enquanto quatro relataram uso superior a um ano, o que indica familiaridade consistente com ferramentas de IA generativa no contexto de desenvolvimento.

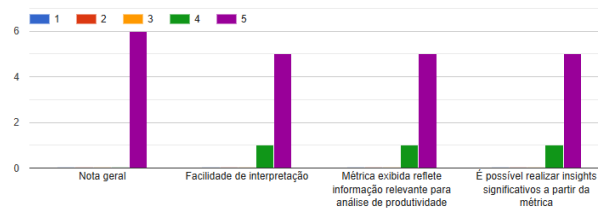


Figura 36 – Distribuição de notas para o gráfico de análise de produtividade por linha de código

6.2.2.2 Avaliação de Usabilidade e Funcionamento

As respostas relacionadas ao funcionamento geral do sistema foram predominantemente positivas.

- **Processo de upload manual:** todos os seis respondentes atribuíram nota 5, indicando que consideraram o procedimento simples e livre de erros.
- **Tempo de resposta do sistema:** novamente, todos os participantes atribuíram nota 5, reconhecendo que as métricas e visualizações são exibidas em tempo adequado.
- **Mensagens de erro e feedback:** houve maior variação — duas notas 3, duas notas 4 e duas nota 5 — sugerindo que, embora funcional, a comunicação de falhas pode ser aprimorada para maior clareza e precisão.

6.2.2.3 Avaliação das Visualizações

Para cada gráfico do *dashboard*, foram geradas representações consolidadas contendo a distribuição das notas nos temas:

- Nota geral
- Facilidade de interpretação
- Relevância da métrica para análise de produtividade
- Potencial de geração de insights

A figura 36 mostra a distribuição de notas para o gráfico de análise de produtividade por linha de código. A figura 37 mostra essa distribuição para o gráfico de análise de produtividade por *commits* e a figura 38 para a correlação de *Pearson*. Além disso, as figuras 39, 40 e 41 mostram, respectivamente, as notas para os gráficos de análise por número de usuários, análise por intensidade de uso do *Copilot* e análise por linguagem de programação.

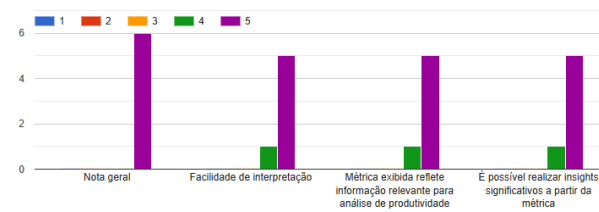


Figura 37 – Distribuição de notas para o gráfico de análise de produtividade por *commits*

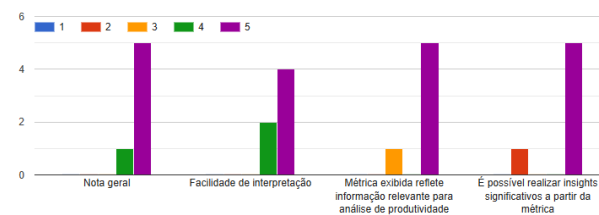


Figura 38 – Distribuição de notas para a correlação de Pearson

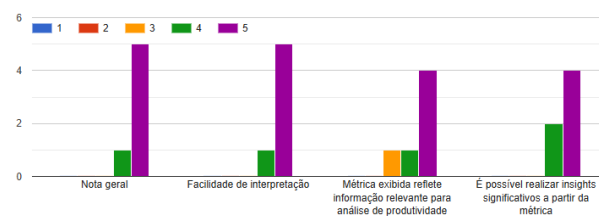


Figura 39 – Distribuição de notas para o gráfico de análise por número de usuários

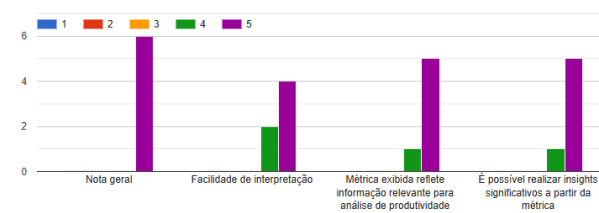


Figura 40 – Distribuição de notas para o gráfico de análise por intensidade de uso do *Copilot*

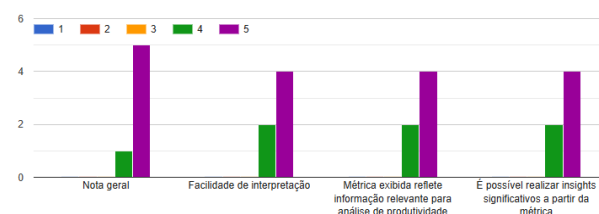


Figura 41 – Distribuição de notas para o gráfico de análise por linguagem de programação

De forma geral, as visualizações receberam avaliações positivas, com predominância de notas 4 e 5. A única métrica que apresentou maior dispersão nas avaliações foi a Correlação de *Pearson*, possivelmente um reflexo de sua natureza mais técnica e da necessidade de maior contextualização estatística para interpretação adequada.

6.2.2.4 Avaliação Qualitativa

A avaliação qualitativa do sistema revelou uma impressão predominantemente positiva entre os validadores. De modo geral, os participantes destacaram que a interface é intuitiva, permitindo navegar pelas funcionalidades com pouca necessidade de instruções adicionais. As visualizações foram frequentemente apontadas como claras e úteis para compreender padrões de uso do *GitHub Copilot*, especialmente quando relacionadas ao desempenho ao longo do tempo. Mesmo entre aqueles que utilizaram dados simulados, houve consenso de que o sistema transmite bem a proposta do projeto e facilita a exploração dos indicadores apresentados.

Os validadores também ressaltaram que a organização das métricas favorece a interpretação dos resultados, contribuindo para a identificação de tendências e comportamentos relevantes. Alguns comentários mencionaram pontos que poderiam ser aprimorados, como a inclusão de explicações adicionais sobre o significado de certos indicadores ou a possibilidade de personalizar filtros. Ainda assim, tais sugestões foram apresentadas mais como oportunidades de expansão do que como críticas à versão atual.

6.2.2.5 Avaliação Geral

De maneira ampla, os resultados da etapa de validação indicam que o sistema alcança satisfatoriamente os objetivos estabelecidos. Os participantes relataram que a plataforma oferece uma visão estruturada do impacto do *GitHub Copilot* sobre a produtividade do time de desenvolvimento, apresentando métricas em um formatos acessíveis e granulares. Mesmo entre aqueles que utilizaram dados simulados, a percepção geral é de que o comportamento do sistema reflete adequadamente cenários reais de uso, permitindo compreender seu potencial em ambientes corporativos.

A organização das informações e a fluidez na navegação foram frequentemente mencionadas como pontos fortes, contribuindo para uma experiência coerente mesmo para usuários que não estavam familiarizados com o projeto. Embora algumas sugestões tenham sido levantadas — como ampliar as possibilidades de filtragem ou adicionar contextualizações mais extensas às métricas — essas observações foram apresentadas como aprimoramentos futuros, e não como limitações críticas da solução.

De forma consolidada, o conjunto das respostas demonstra que o sistema apresenta aderência ao propósito de analisar o uso do *GitHub Copilot* em diferentes contextos. A experiência relatada pelos validadores, tanto com dados reais quanto com conjuntos

simulados, reforça que a ferramenta possui potencial para apoiar decisões e facilitar o entendimento de padrões de utilização em equipes de desenvolvimento.

6.3 Resumo

A análise conjunta das avaliações revela um panorama consistente sobre o desempenho e a utilidade do sistema desenvolvido. Apesar das limitações impostas pelas políticas internas de segurança das empresas, que restringiram o acesso a informações reais para a maioria dos validadores, as respostas obtidas indicam que a solução apresenta maturidade para apoiar decisões em relação ao uso do *GitHub Copilot*.

O validador que operou o sistema com dados reais forneceu um conjunto de observações particularmente valioso, confirmando que as funcionalidades centrais, como a visualização de métricas e o envio automático de *reports*, funcionam conforme o esperado. É destacado o potencial do sistema como ferramenta de apoio a compreensão da adoção do *Copilot* em ambientes corporativos, sinalizando apenas necessidades pontuais de aprimoramento.

Os participantes que utilizaram dados simulados apresentaram percepções amplamente convergentes às do validador de dados reais. Tanto a interface quanto a organização das visualizações foram frequentemente elogiadas, reforçando a boa usabilidade geral. As notas atribuídas aos gráficos evidenciam uma recepção majoritariamente positiva, com maior dispersão observada apenas na métrica de correlação de *Pearson* — resultado compreensível, dada sua natureza mais dependente de contextualização estatística.

De forma geral, os resultados obtidos nas duas frentes de avaliação convergem para a conclusão de que o sistema cumpre seu propósito e demonstra potencial de uso em cenários reais. As respostas reforçam que a solução oferece um conjunto de visualizações informativas, apresenta desempenho adequado e possui aderência aos objetivos definidos pelo projeto. As sugestões feitas pelos validadores apontam caminhos naturais de evolução, mas não comprometem a percepção de utilidade, indicando que o sistema já se encontra em um estágio sólido de desenvolvimento.

7 Considerações Finais

Este capítulo abrevia os resultados obtidos com a implementação, sintetiza os aprendizados decorrentes do processo de desenvolvimento e descreve direções potenciais para a expansão da plataforma.

7.1 Visão Geral

O sistema desenvolvido teve como objetivo construir uma plataforma capaz de integrar dados provenientes de repositórios *Git* e informações de uso do *GitHub Copilot*, de modo a permitir a análise quantitativa da produtividade de equipes de desenvolvimento. Esse objetivo foi alcançado por meio da implementação de uma solução que realiza a automatização da coleta, consolidação e visualização desses dados em um ambiente unificado, acessível via interface web.

A implementação apresentada no Capítulo 5 materializou os requisitos estabelecidos no Capítulo 4, resultando em um sistema funcional que organiza métricas de produtividade e uso da IA de forma integrada. Ainda que a plataforma não tenha como propósito fornecer conclusões definitivas sobre o impacto absoluto de ferramentas de IA na produtividade, ela cria condições objetivas para que tais relações possam ser exploradas de maneira sistemática, contextualizada e apoiada em evidências.

Os módulos de visualização desenvolvidos oferecem elementos analíticos que permitem observar padrões de trabalho, variações temporais, dinâmicas de adoção do *Copilot* e potenciais correlações entre uso da IA e indicadores de produção de código. Dessa forma, o sistema consolida uma base concreta para estudos posteriores, aprimoramentos da plataforma e investigações mais amplas sobre o papel de assistentes baseados em modelos de linguagem no processo de desenvolvimento de software.

7.2 Trabalhos Futuros

A continuidade deste trabalho pode ocorrer em diferentes direções, especialmente no que se refere à ampliação da coleta de dados, ao desenvolvimento de novos módulos analíticos e à integração da plataforma com outras ferramentas utilizadas em processos reais de engenharia de software.

Um primeiro eixo de evolução envolve a ampliação da coleta de dados relacionados ao uso de ferramentas de inteligência artificial generativa. Atualmente, a plataforma consome exclusivamente métricas fornecidas pelo *GitHub Copilot*, porém seria desejável incorporar

dados provenientes de outras ferramentas de IA. Essa expansão, entretanto, depende diretamente do modo como cada fornecedor disponibiliza informações sobre o uso de suas soluções, bem como do nível de granularidade exposto. Especificações mais detalhadas sobre o uso da IA poderiam permitir análises mais abrangentes. Além dos dados de interação com a IA, a plataforma poderia ser estendida para coletar informações adicionais relacionadas ao processo de desenvolvimento (relacionados a vazão de produtividade), como histórico de revisões de *pull requests* (PRs). Neste último exemplo, ainda, seria possível começar uma discussão acerca de outros aspectos de código além de vazão: a partir da análise de PRs poderia-se estipular a evolução da qualidade de desenvolvimento de código (se muitos PRs têm sido rejeitados, a equipe de desenvolvimento tem problemas).

A partir da incorporação de novos tipos de dados, torna-se possível elaborar métricas mais variadas. Indicadores que relacionem uso da IA com variações no esforço de revisão ou na complexidade do código produzido, por exemplo, podem enriquecer a capacidade analítica do sistema. Essa ampliação permitiria investigar, de forma mais diversificada, como diferentes dimensões do processo de desenvolvimento são influenciadas pelo uso contínuo de LLMs.

Outra possibilidade de avanço reside na generalização da arquitetura de coleta de dados. No estado atual, a plataforma é integrada exclusivamente ao *GitHub*; contudo, uma extensão natural é o suporte a outros provedores de versionamento e colaboração, como *GitLab* ou *Bitbucket*. Essa generalização promoveria uma democratização da plataforma desenvolvida para organizações que não utilizam exclusivamente o ecossistema *GitHub*.

No âmbito dos módulos analíticos, há espaço para aprofundamento teórico. Técnicas de análise temporal, aprendizado de máquina e métodos estatísticos poderiam ser utilizadas para identificar tendências e variações sistemáticas na produtividade das equipes, relacionada ao uso de ferramentas de IA. Atualmente, a plataforma somente tem capacidade analítica muito limitada: opera, sobretudo, como um *dashboard* para organizar os dados e colocá-los na tela, mas quem de fato realiza a análise é o próprio usuário. A ideia seria munir o sistema com ferramentas analíticas para facilitar este processo.

A integração da plataforma com sistemas de gerenciamento de tarefas, como *Jira*, *Trello* ou *Slack*, constitui outra direção possível. A incorporação de informações relativas a planejamento, alocação e conclusão de atividades permitiria analisar o desempenho das equipes sob a ótica de metas e entregas. Essa integração abriria espaço para investigar, por exemplo, se padrões de uso da IA estão associados a melhorias dos *KPIs*¹ acompanhados pela equipe de desenvolvimento.

Por fim, a plataforma pode ser aprimorada em termos de personalização. Um exemplo evidente disso seria a *feature* de relatórios automáticos. Seria ideal que o usuário

¹ *Key Performance Indicators* são métricas mensuráveis que avaliam o sucesso de uma empresa, projeto ou processo em relação a objetivos estratégicos.

conseguisse configurar, além da periodicidade, outros elementos relevantes acerca do *report*, por exemplo, quais gráficos a serem incluídos no email.

Referências

BUSSAB, W. de O.; MORETTIN, P. A. *Estatística Básica*. 6. ed. São Paulo, SP: Editora Saraiva, 2010. Acesso em: 22 abril 2025.

Cisco Outshift. *Understanding LLMs: Model Size, Training Data, and Tokenization*. 2024. Blog da Cisco Outshift. Disponível em: <<https://outshift.cisco.com/blog/understanding-llms-model-size-training-data-tokenization>>. Acesso em: 3 December 2025.

DEEPLARNINGAI. *Python for Data Analysis: Key Stats and Trends*. 2024. Disponível em: <<https://www.deeplearning.ai/blog/python-for-data-analysis-key-stats-and-trends/>>.

DONG, Y. et al. A survey on code generation with llm-based agents. 2025. Disponível em: <<https://arxiv.org/html/2508.00083v1#S9>>. Acesso em: 3 December 2025.

GITHUB, I. *GitHub Copilot documentation*. [S.l.], 2025. Disponível em: <<https://github.com/features/copilot>>.

GNANASAMBANDAM, C.; HARRYSSON, M. *Yes, you can measure software developer productivity*. [S.l.], 2023. Accessed on September 27, 2025. Disponível em: <https://www.mckinsey.com.br/industries/technology-media-and-telecommunications/our-insights/yes-you-can-measure-software-developer-productivity?utm_source=chatgpt.com>.

HARNESS. *The State of Software Delivery*. [S.l.], 2025. Disponível em: <<https://www.harness.io/state-of-software-delivery>>.

HOFFMAN, P. *AI's Power Demand: Calculating ChatGPT's electricity consumption for handling over 365 billion user queries every year*. [S.l.], 2025. Disponível em: <<https://www.bestbrokers.com/forex-brokers/ais-power-demand-calculating-chatgpts-electricity-consumption-for-handling-over-78-billion-user-queries>>.

HUNNER, T. *LBYL vs. EAFP: Comparing Python Programming Styles*. 2020. Real Python. Disponível em: <<https://realpython.com/python-lbyl-vs-eafp/>>. Acesso em: 3 dez. 2025.

SCACCHI, W. Understanding software productivity. *Advances in Software Engineering and Knowledge Engineering*, v. 4, p. 37–70, 1994.

STREIT, J.; FEYE, L. *Benchmarking Ongoing Development Output in Real-Life Software Projects*. [S.l.], 2024. Disponível em: <https://link.springer.com/chapter/10.1007/978-3-031-78392-0_2>.

TABARSI, B. et al. Llms' reshaping of people, processes, products, and society in software development: A comprehensive exploration with early adopters. 2025. Disponível em: <<https://arxiv.org/abs/2503.05012>>. Acesso em: 3 December 2025.

WEAVER, D. *AI Case Study: PwC Invests \$1B, Increases Productivity 40% With Generative AI*. [S.l.], 2024. Accessed on September 27, 2025. Disponível em: <<https://ai-for-business/ai-case-study-pwc-commits-1b-increases-productivity-40-with-generative-ai/>>.

WORKLYTICS, I. *Calculating the ROI of Generative-AI Tools: A Practical Framework Leveraging Worklytics Metrics*. [S.l.], 2025. Disponível em: <<https://www.worklytics.co/resources/calculating-roi-generative-ai-tools-worklytics-framework>>.

ZAROS, L. G.; MEDEIROS, H. R. de. *Bioestatística*. 2. ed. Natal, RN: Editora da UFRN, 2011. Acesso em: 22 abril 2025.